

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI



Đỗ Quang Hợp

**NGHIÊN CỨU VÀ XÂY DỰNG NỀN TẢNG
EDGE AI TÍCH HỢP AI AGENT VÀ TỐI ƯU
MÔ HÌNH YOLO**

*OPTIMIZING YOLO FOR EDGE AI AND DEVELOPING AN EDGE-AGENT
MANAGEMENT SYSTEM WITH DASHBOARD AND AI AGENT*

ĐỒ ÁN TỐT NGHIỆP

Ngành: Khoa Học Máy Tính

Khoa: Khoa Công Nghệ Thông Tin

HÀ NỘI – 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI



Đỗ Quang Hợp

**NGHIÊN CỨU VÀ XÂY DỰNG NỀN TẢNG EDGE
AI TÍCH HỢP AI AGENT VÀ TỐI ƯU MÔ HÌNH
YOLO**

*OPTIMIZING YOLO FOR EDGE AI AND DEVELOPING AN EDGE-AGENT
MANAGEMENT SYSTEM WITH DASHBOARD AND AI AGENT*

ĐỒ ÁN TỐT NGHIỆP

Ngành: Khoa Học Máy Tính

Khoa: Khoa Công Nghệ Thông Tin

Giảng viên hướng dẫn: TS. Hoàng Nam Thắng

Sinh viên thực hiện: Đỗ Quang Hợp

Mã số sinh viên: 68CNCS

Lớp: 68CS2

HÀ NỘI – 2026

*Trang này dùng để dán bản Nhận xét của người hướng dẫn, hoặc thay bằng bản
nhận xét đã ký.*

*Trang này dùng để dán bản Nhận xét của người phản biện, hoặc thay bằng bản
nhận xét đã ký.*

TÓM TẮT

Tên đề tài: NGHIÊN CỨU VÀ XÂY DỰNG NỀN TẢNG EDGE AI TÍCH HỢP AI AGENT VÀ TỐI ƯU MÔ HÌNH YOLO

Sinh viên thực hiện: Đỗ Quang Hợp

Mã SV: 68CNCS

Lớp: 68CS2

Các mô hình phát hiện đối tượng thuộc họ YOLO có ưu điểm về tốc độ, nhưng khi triển khai trên thiết bị Edge AI vẫn phải đáp ứng đồng thời các ràng buộc về độ chính xác, độ trễ, bộ nhớ, điện năng và khả năng vận hành ổn định. Bên cạnh tối ưu mô hình, việc quản lý nhiều thiết bị phân tán còn đòi hỏi một cơ chế thống nhất để đăng ký thiết bị, phân phối mô hình, cập nhật cấu hình, theo dõi sức khỏe và xử lý sự cố. Đề án này đề xuất một giải pháp tích hợp gồm quy trình tối ưu mô hình YOLO được lựa chọn cho thiết bị Edge AI mục tiêu và một hệ thống quản lý tập trung các Edge Agent. Quy trình tối ưu bao gồm chuẩn hóa mô hình, lựa chọn kích thước đầu vào, rút gọn mô hình khi phù hợp, lượng tử hóa, biên dịch theo bộ tăng tốc và hiệu chỉnh các bước tiền xử lý–hậu xử lý. Hệ thống quản lý được tổ chức theo mô hình control plane–edge plane, trong đó mỗi Edge Agent duy trì kênh liên lạc an toàn với máy chủ, nhận lệnh có phiên bản, báo cáo telemetry và hỗ trợ khôi phục khi triển khai thất bại. Dashboard cung cấp góc nhìn tổng hợp về trạng thái thiết bị, phiên bản mô hình và cảnh báo. AI Agent tiếp nhận yêu cầu ngôn ngữ tự nhiên, truy vấn dữ liệu vận hành và tạo kế hoạch thao tác thông qua tập công cụ được giới hạn quyền; các hành động có ảnh hưởng cao phải được người dùng phê duyệt trước khi thực thi. Giải pháp được đánh giá theo ba nhóm tiêu chí: chất lượng nhận diện, hiệu năng trên thiết bị biên và khả năng vận hành hệ thống. Kết quả định lượng cuối cùng cần được cập nhật từ thí nghiệm thực tế, gồm mAP, độ trễ, FPS, mức sử dụng tài nguyên, tỷ lệ triển khai thành công và thời gian phản hồi lệnh. Đóng góp chính của đề án là kết hợp tối ưu suy luận với quản trị vòng đời mô hình và thiết bị trong một kiến trúc có khả năng quan sát, kiểm soát và mở rộng.

Trang này dùng để dán bản Nhiệm vụ đồ án tốt nghiệp, hoặc thay bằng bản nhiệm vụ đã được phê duyệt.

LỜI NÓI ĐẦU

Sự phát triển nhanh của trí tuệ nhân tạo tại biên đang mở ra khả năng xử lý dữ liệu hình ảnh trực tiếp tại nơi dữ liệu được sinh ra. Cách tiếp cận này giúp giảm độ trễ, hạn chế lưu lượng truyền về máy chủ và tăng khả năng hoạt động khi kết nối mạng không ổn định. Tuy nhiên, việc đưa mô hình phát hiện đối tượng YOLO lên thiết bị có tài nguyên hạn chế không chỉ là bài toán tối ưu mô hình. Một hệ thống thực tế còn cần cơ chế triển khai phiên bản, giám sát trạng thái, thu thập chỉ số, điều khiển từ xa và hỗ trợ người vận hành ra quyết định.

Từ bối cảnh đó, đề án lựa chọn đề tài “Tối ưu mô hình YOLO cho Edge AI và xây dựng hệ thống quản lý Edge Agent tích hợp Dashboard và AI Agent”. Nội dung báo cáo tập trung vào hai phần có quan hệ chặt chẽ: tối ưu chuỗi suy luận để mô hình đáp ứng ràng buộc của thiết bị biên; đồng thời thiết kế nền tảng quản lý tập trung cho phép theo dõi, cấu hình và điều phối nhiều Edge Agent. AI Agent được đặt trong lớp điều khiển có kiểm soát, nhằm chuyển yêu cầu ngôn ngữ tự nhiên thành kế hoạch và lệnh hệ thống có thể kiểm tra, phê duyệt và truy vết.

Trong quá trình thực hiện đề án, em đã nhận được sự hướng dẫn và hỗ trợ của nhiều cá nhân, tập thể. Em xin trân trọng cảm ơn TS. Hoàng Nam Thắng đã định hướng chuyên môn, góp ý phương pháp thực hiện và dành thời gian nhận xét các phiên bản của đề án. Em cảm ơn các thầy cô của Trường Đại học Xây dựng Hà Nội đã trang bị kiến thức nền tảng trong suốt quá trình học tập. Em cũng xin cảm ơn gia đình, bạn bè và các thành viên đã hỗ trợ thử nghiệm hệ thống. Mặc dù đã cố gắng hoàn thiện, báo cáo khó tránh khỏi thiếu sót; em mong nhận được ý kiến đóng góp để tiếp tục cải tiến kết quả nghiên cứu.

Hà Nội, ngày [CẦN BỔ SUNG: ngày] tháng [CẦN BỔ SUNG: tháng] năm 2026

Sinh viên thực hiện

Đỗ Quang Hợp

CAM ĐOAN

Tôi xin cam đoan đồ án tốt nghiệp này là kết quả học tập, nghiên cứu và triển khai của bản thân dưới sự hướng dẫn của TS. Hoàng Nam Thắng. Các nội dung tham khảo, số liệu và kết quả của công trình khác được sử dụng trong báo cáo đều được trích dẫn nguồn rõ ràng. Các hình vẽ, bảng biểu và kết quả thực nghiệm do tôi tạo ra được trình bày trung thực; những dữ liệu chưa được đo kiểm được đánh dấu để bổ sung, không được sử dụng như kết quả đã xác nhận.

Tôi chịu trách nhiệm về tính trung thực, tính nguyên bản và việc tuân thủ quy định liên chính học thuật của toàn bộ nội dung đồ án. Nếu phát hiện có hành vi sao chép, giả mạo số liệu hoặc sử dụng kết quả của người khác mà không trích dẫn phù hợp, tôi xin hoàn toàn chịu trách nhiệm theo quy định của Nhà trường.

Hà Nội, ngày [CẦN BỔ SUNG: ngày] tháng [CẦN BỔ SUNG: tháng] năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Đỗ Quang Hợp

Mục lục

LỜI NÓI ĐẦU	i
LỜI CAM ĐOAN	ii
MỤC LỤC	iii
DANH SÁCH HÌNH VẼ	vi
DANH SÁCH BẢNG	vii
DANH SÁCH CÁC KÝ HIỆU, CHỮ VIẾT TẮT	viii
MỞ ĐẦU	1
1 TỔNG QUAN BÀI TOÁN VÀ YÊU CẦU HỆ THỐNG	3
1.1 Bối cảnh bài toán	3
1.2 Phát biểu bài toán	4
1.3 Các bên liên quan và kịch bản sử dụng	4
1.4 Yêu cầu chức năng	5
1.5 Yêu cầu phi chức năng	5
1.6 Các hướng tiếp cận hiện có và khoảng trống	6
1.7 Đóng góp của đề án	6
1.8 Kết chương	7
2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ	8
2.1 Mô hình phát hiện đối tượng YOLO	8
2.1.1 Giao trên hợp và hậu xử lý	8
2.1.2 Chỉ số chất lượng nhận diện	9
2.2 Tối ưu mô hình cho thiết bị biên	9
2.2.1 Lượng tử hóa	10
2.2.2 Cắt tỉa và chưng cất tri thức	11
2.2.3 Biên dịch đồ thị và bộ thực thi	11

2.3	Đặc trưng của Edge AI	11
2.4	Quản lý Edge Agent và vòng đời mô hình	12
2.5	Dashboard, telemetry và cảnh báo	12
2.6	AI Agent sử dụng công cụ có kiểm soát	13
2.7	Kết chương	13
3	THIẾT KẾ GIẢI PHÁP ĐỀ XUẤT	14
3.1	Nguyên tắc thiết kế	14
3.2	Kiến trúc tổng thể	14
3.2.1	Control plane	15
3.2.2	Edge plane	15
3.3	Mô hình dữ liệu	16
3.4	Quy trình tối ưu và đóng gói mô hình	16
3.5	Vòng đời thiết bị và triển khai	17
3.6	Giao thức lệnh và chống thực thi trùng	18
3.7	Thiết kế Dashboard	18
3.8	Thiết kế AI Agent điều khiển hệ thống	18
3.9	An toàn, bảo mật và riêng tư	19
3.10	Kết chương	19
4	CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG	21
4.1	Môi trường và tổ chức mã nguồn	21
4.2	Hiện thực Edge Agent	22
4.2.1	Quản lý tiến trình suy luận	23
4.2.2	Cập nhật mô hình nguyên tử	23
4.2.3	Telemetry và bộ đệm ngoại tuyến	24
4.3	Hiện thực control plane	24
4.3.1	API quản lý	24
4.3.2	Xử lý heartbeat và reported state	25
4.3.3	Bộ điều phối triển khai	25
4.4	Hiện thực Dashboard	26
4.5	Hiện thực AI Agent	26

4.6	Đóng gói và triển khai	27
4.7	Kiểm thử cài đặt	27
4.8	Kết chương	27
5	THỰC NGHIỆM VÀ ĐÁNH GIÁ	28
5.1	Môi trường thực nghiệm	28
5.2	Dữ liệu và mô hình cơ sở	28
5.3	Chỉ số đánh giá	29
5.4	Quy trình đo hiệu năng mô hình	29
5.5	Kết quả tối ưu mô hình	30
5.6	Đánh giá chuỗi tiền xử lý và hậu xử lý	31
5.7	Đánh giá hệ thống quản lý Edge Agent	31
5.8	Đánh giá AI Agent	33
5.9	Thảo luận kết quả	34
5.10	Các nguy cơ ảnh hưởng tính hợp lệ	34
5.11	Kết chương	34
	KẾT LUẬN	36
	TÀI LIỆU THAM KHẢO	38
A	GIAO DIỆN LẬP TRÌNH VÀ CẤU HÌNH THAM CHIẾU	40
A.1	Cấu trúc manifest của gói mô hình	40
A.2	Ví dụ bản tin heartbeat	41
A.3	Ví dụ tạo kế hoạch triển khai	41
A.4	Ma trận quyền tham khảo	42
B	NHẬT KÝ THỰC HIỆN ĐATN	43

Danh sách hình vẽ

3.1	Kiến trúc tổng thể của giải pháp đề xuất	15
3.2	Quy trình tối ưu và phát hành mô hình	17

Danh sách bảng

1.1	Yêu cầu chức năng của hệ thống	5
2.1	Các nhóm kỹ thuật tối ưu mô hình	10
3.1	Các thực thể dữ liệu chính	16
4.1	Công nghệ sử dụng trong phiên bản cài đặt	22
4.2	Các nhóm API quản lý chính	25
5.1	Cấu hình phần cứng và phần mềm thực nghiệm	28
5.2	Thống kê bộ dữ liệu	29
5.3	Ảnh hưởng của tối ưu đến chất lượng nhận diện	30
5.4	Hiệu năng mô hình trên thiết bị Edge AI	30
5.5	Phân rã độ trễ chuỗi xử lý	31
5.6	Kết quả kiểm thử kịch bản vận hành	32
5.7	Kết quả kiểm thử tải control plane	33
5.8	Kết quả đánh giá AI Agent	33
5.9	Ma trận lựa chọn cấu hình triển khai	34
A.1	Ma trận quyền theo vai trò	42

DANH SÁCH CÁC KÝ HIỆU, CHỮ VIẾT TẮT

Ký hiệu	Ý nghĩa
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface – Giao diện lập trình ứng dụng
CPU	Central Processing Unit – Bộ xử lý trung tâm
GPU	Graphics Processing Unit – Bộ xử lý đồ họa
NPU	Neural Processing Unit – Bộ xử lý mạng nơ-ron
Edge AI	Thực thi tác vụ trí tuệ nhân tạo trên hoặc gần nguồn sinh dữ liệu
FPS	Frames Per Second – Số khung hình xử lý trong một giây
IoU	Intersection over Union – Tỷ lệ giao trên hợp
mAP	mean Average Precision – Độ chính xác trung bình trên các lớp
MQTT	Message Queuing Telemetry Transport – Giao thức truyền thông điệp nhẹ
ONNX	Open Neural Network Exchange – Định dạng trao đổi mô hình học máy
PTQ	Post-Training Quantization – Lượng tử hóa sau huấn luyện
QAT	Quantization-Aware Training – Huấn luyện có nhận biết lượng tử hóa
RAM	Random Access Memory – Bộ nhớ truy cập ngẫu nhiên
RBAC	Role-Based Access Control – Kiểm soát truy cập theo vai trò
REST	Representational State Transfer – Phong cách kiến trúc dịch vụ web
TLS	Transport Layer Security – Giao thức bảo mật tầng truyền tải
YOLO	You Only Look Once – Họ mô hình phát hiện đối tượng một giai đoạn

MỞ ĐẦU

1. Lý do lựa chọn đề tài

Nhiều ứng dụng như giám sát công trường, kiểm tra an toàn, giao thông thông minh và tự động hóa công nghiệp cần phát hiện đối tượng với thời gian phản hồi ngắn. Nếu toàn bộ luồng hình ảnh được truyền tới máy chủ đám mây, hệ thống có thể phát sinh độ trễ, chi phí băng thông và rủi ro lộ dữ liệu. Edge AI giải quyết một phần các hạn chế này bằng cách đưa mô hình suy luận tới gần camera hoặc cảm biến. Tuy nhiên, thiết bị biên thường có năng lực tính toán, bộ nhớ và công suất giới hạn, trong khi mô hình phát hiện đối tượng phải duy trì chất lượng đủ tốt trong nhiều điều kiện môi trường.

Trong thực tế, tối ưu một tập mô hình chưa đủ tạo nên hệ thống có thể vận hành. Khi số lượng thiết bị tăng, người quản trị phải biết thiết bị nào đang trực tuyến, đang chạy phiên bản mô hình nào, có đạt ngưỡng hiệu năng hay không và cần xử lý thế nào khi triển khai lỗi. Vì vậy, đề án tiếp cận bài toán ở cả hai tầng: tối ưu chuỗi suy luận YOLO trên Edge AI và xây dựng nền tảng quản lý tập trung cho các Edge Agent. Dashboard phục vụ quan sát trực quan; AI Agent hỗ trợ truy vấn và điều khiển bằng ngôn ngữ tự nhiên nhưng phải tuân theo chính sách quyền hạn và cơ chế phê duyệt.

2. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề án là xây dựng giải pháp giúp triển khai và vận hành mô hình YOLO hiệu quả trên một nhóm thiết bị Edge AI. Mục tiêu này được cụ thể hóa thành: (i) xây dựng quy trình tối ưu có thể lặp lại và so sánh được giữa mô hình cơ sở với các biến thể tối ưu; (ii) thiết kế Edge Agent có khả năng chạy suy luận, tự giám sát và nhận cấu hình từ xa; (iii) xây dựng control plane quản lý thiết bị, phiên bản mô hình, lệnh và telemetry; (iv) xây dựng Dashboard theo dõi và điều khiển; (v) tích hợp AI Agent để hỗ trợ truy vấn, chẩn đoán và tạo kế hoạch thao tác có kiểm soát; và (vi) đánh giá giải pháp bằng chỉ số nhận diện, hiệu năng thiết bị và chất lượng vận hành.

3. Đối tượng và phạm vi

Đối tượng nghiên cứu gồm mô hình phát hiện đối tượng YOLO, kỹ thuật tối ưu suy luận, thiết bị tăng tốc AI tại biên và kiến trúc quản lý thiết bị phân tán. Phạm vi thực nghiệm sử dụng bộ dữ liệu của đề tài, mô hình YOLO được lựa chọn và thiết bị Edge AI mục tiêu. Báo cáo tập trung vào suy luận hình ảnh hoặc video; không đi sâu vào thiết kế phần cứng tăng tốc và không thay thế một nền tảng MLOps đầy đủ ở quy mô trung tâm dữ liệu. AI Agent chỉ được phép gọi các công cụ do hệ thống công bố, không được tự ý thực thi lệnh hệ điều hành hoặc bỏ qua bước phê duyệt đối với thao tác có rủi ro.

4. Phương pháp thực hiện

Đề án sử dụng phương pháp thực nghiệm kết hợp thiết kế hệ thống. Trước hết, mô hình cơ sở được đo trên cùng bộ dữ liệu và cấu hình thiết bị để hình thành đường chuẩn. Tiếp theo, từng kỹ thuật tối ưu được áp dụng có kiểm soát, ghi nhận ảnh hưởng đến độ chính xác, độ trễ, thông lượng, bộ nhớ và công suất. Song song với đó, hệ thống quản lý được phân rã thành các thành phần, mô tả giao thức, dữ liệu và chính sách an toàn trước khi cài đặt. Các kịch bản vận hành như mất kết nối, cập nhật lỗi, khôi phục phiên bản và phê duyệt lệnh AI Agent được kiểm thử để đánh giá độ tin cậy.

5. Bố cục báo cáo

Ngoài phần mở đầu, kết luận, tài liệu tham khảo và phụ lục, báo cáo gồm năm chương. Chương I xác định bài toán, các bên liên quan và yêu cầu hệ thống. Chương II trình bày cơ sở về YOLO, tối ưu mô hình, Edge AI, quản lý thiết bị và AI Agent. Chương III đề xuất kiến trúc tổng thể, quy trình tối ưu và các luồng điều khiển. Chương IV mô tả cài đặt các thành phần và phương án triển khai. Chương V xây dựng phương pháp thực nghiệm, bảng kết quả và tiêu chí đánh giá giải pháp.

CHƯƠNG I. TỔNG QUAN BÀI TOÁN VÀ YÊU CẦU HỆ THỐNG

Chương này phân tích bối cảnh triển khai phát hiện đối tượng trên thiết bị biên, xác định khoảng trống giữa một mô hình chạy được trong phòng thí nghiệm và một hệ thống có thể vận hành nhiều thiết bị. Trên cơ sở đó, chương xác lập các bên liên quan, kịch bản sử dụng, yêu cầu chức năng, yêu cầu phi chức năng và các đóng góp dự kiến của đề án.

1.1 Bối cảnh bài toán

Một hệ thống thị giác máy tính tập trung thường thu nhận hình ảnh tại camera, truyền dữ liệu qua mạng và thực hiện suy luận tại máy chủ. Kiến trúc này thuận tiện cho quản trị tài nguyên tính toán nhưng phụ thuộc nhiều vào kết nối, tạo tải băng thông lớn và làm tăng thời gian phản hồi. Đối với các tình huống yêu cầu cảnh báo nhanh, dữ liệu nhạy cảm hoặc khu vực có mạng không ổn định, việc xử lý tại biên là lựa chọn phù hợp hơn. Mô hình được triển khai trên máy tính công nghiệp, thiết bị nhúng hoặc bộ tăng tốc đặt gần nguồn dữ liệu; chỉ kết quả nhận diện, sự kiện và telemetry cần thiết được gửi về trung tâm.

YOLO là họ mô hình phát hiện đối tượng một giai đoạn, có khả năng dự đoán vị trí và lớp đối tượng trong một lần lan truyền thuận. Đặc điểm này giúp YOLO phù hợp với ứng dụng thời gian thực. Dù vậy, mô hình huấn luyện trên máy trạm không thể mặc nhiên đạt hiệu năng mong muốn khi chuyển sang thiết bị biên. Sự khác biệt về phần cứng, thư viện, phép toán được hỗ trợ và băng thông bộ nhớ có thể làm tăng độ trễ hoặc gây lỗi chuyển đổi. Các kỹ thuật giảm kích thước mô hình cũng có thể làm suy giảm chất lượng nhận diện, đặc biệt với vật thể nhỏ hoặc lớp có ít mẫu.

Bài toán vận hành xuất hiện khi một mô hình phải được triển khai trên nhiều thiết bị. Quản trị viên cần nắm được danh tính, cấu hình, trạng thái kết nối, phiên bản phần mềm và phiên bản mô hình của từng Edge Agent. Mỗi thay đổi cần có lịch sử, kết quả và khả năng khôi phục. Vì vậy, đối tượng tối ưu của đề án không chỉ là mạng nơ-ron mà là toàn bộ chuỗi từ dữ liệu đầu vào, tiền xử lý, suy luận, hậu xử lý đến phân phối phiên bản và giám sát trong quá trình chạy.

1.2 Phát biểu bài toán

Đầu vào của bài toán gồm một mô hình YOLO cơ sở đã được huấn luyện, bộ dữ liệu đánh giá, cấu hình thiết bị Edge AI mục tiêu và một nhóm thiết bị cần quản lý. Đầu ra thứ nhất là gói mô hình tối ưu có thông tin phiên bản, cấu hình tiền xử lý–hậu xử lý và kết quả kiểm chứng. Đầu ra thứ hai là hệ thống quản lý cho phép đăng ký Edge Agent, triển khai gói mô hình, cập nhật cấu hình, theo dõi telemetry, phát hiện bất thường và thực hiện lệnh từ xa. Đầu ra thứ ba là giao diện Dashboard và AI Agent giúp người dùng quan sát, truy vấn và điều phối hệ thống.

Giải pháp cần tìm một điểm cân bằng giữa chất lượng nhận diện và chi phí thực thi. Có thể biểu diễn mục tiêu lựa chọn cấu hình x dưới dạng:

$$\max_x U(x) = w_a A(x) - w_l L(x) - w_m M(x) - w_p P(x), \quad (1.1)$$

trong đó A là chất lượng nhận diện, L là độ trễ, M là bộ nhớ đỉnh, P là công suất và các hệ số w thể hiện mức ưu tiên của ứng dụng. Bài toán còn chịu các ràng buộc như độ trễ không vượt ngưỡng, mức sử dụng bộ nhớ nằm trong khả năng thiết bị và độ chính xác không giảm quá giới hạn được chấp nhận. Các hệ số và ngưỡng phải được xác định từ yêu cầu thực tế, không chọn tùy ý sau khi đã quan sát kết quả.

1.3 Các bên liên quan và kịch bản sử dụng

Hệ thống có bốn nhóm tác nhân chính. Quản trị viên chịu trách nhiệm đăng ký thiết bị, phân quyền, phát hành phiên bản và xử lý sự cố. Người vận hành theo dõi Dashboard, xác nhận cảnh báo và thực hiện các thao tác thường ngày. Kỹ sư mô hình chuẩn bị, kiểm thử và công bố gói mô hình. Edge Agent chạy trên thiết bị biên, thực hiện suy luận và báo cáo trạng thái. AI Agent là thành phần hỗ trợ người dùng chứ không phải chủ thể có quyền không giới hạn.

Các kịch bản nghiệp vụ cốt lõi gồm: (i) thiết bị mới đăng ký và nhận danh tính; (ii) quản trị viên gán mô hình cho một thiết bị hoặc nhóm thiết bị; (iii) Edge Agent tải, kiểm tra, kích hoạt phiên bản rồi gửi kết quả; (iv) Dashboard hiển thị trạng thái, chỉ số và cảnh báo; (v) người vận hành yêu cầu AI Agent tổng hợp tình trạng hoặc đề xuất cách xử lý; (vi) AI Agent tạo kế hoạch lệnh, hệ thống kiểm tra quyền và yêu cầu phê duyệt nếu cần; (vii) thiết bị tự khôi phục về phiên bản ổn định khi bản cập nhật không vượt qua kiểm tra sức khỏe.

1.4 Yêu cầu chức năng

Bảng 1.1 trình bày các yêu cầu chức năng tối thiểu. Mã yêu cầu được dùng để liên kết thiết kế, cài đặt và kiểm thử trong các chương sau.

Bảng 1.1: Yêu cầu chức năng của hệ thống

Mã	Nhóm chức năng	Mô tả
FR01	Quản lý danh tính	Đăng ký, xác thực, vô hiệu hóa và gắn nhãn Edge Agent theo vị trí, phần cứng hoặc mục đích sử dụng.
FR02	Quản lý mô hình	Lưu siêu dữ liệu, tệp mô hình, hàm băm, phiên bản, cấu hình đầu vào và điều kiện tương thích.
FR03	Triển khai	Gán phiên bản cho thiết bị hoặc nhóm thiết bị; theo dõi tiến độ tải, kiểm tra, kích hoạt và khôi phục.
FR04	Cấu hình	Cập nhật ngưỡng tin cậy, kích thước đầu vào, nguồn video, tần suất báo cáo và chính sách lưu sự kiện.
FR05	Telemetry	Thu thập nhịp sống, nhiệt độ, mức sử dụng CPU/GPU/NPU, bộ nhớ, FPS, độ trễ và số lỗi.
FR06	Cảnh báo	Sinh cảnh báo khi thiết bị mất kết nối, nhiệt độ cao, hiệu năng suy giảm, lỗi suy luận hoặc triển khai thất bại.
FR07	Dashboard	Hiển thị tổng quan, danh sách thiết bị, chi tiết thiết bị, phiên bản mô hình, lịch sử lệnh và biểu đồ chỉ số.
FR08	Điều khiển	Cho phép khởi động lại dịch vụ suy luận, cập nhật cấu hình, triển khai hoặc khôi phục phiên bản theo quyền.
FR09	AI Agent	Trả lời truy vấn vận hành, tổng hợp nguyên nhân, tạo kế hoạch và gọi tập công cụ được cấp phép.
FR10	Kiểm toán	Lưu người khởi tạo, nội dung, thời gian, trạng thái và kết quả của mọi thao tác làm thay đổi hệ thống.

1.5 Yêu cầu phi chức năng

Về hiệu năng, Edge Agent phải duy trì mục tiêu FPS và độ trễ được xác định cho kịch bản ứng dụng; control plane phải xử lý được số lượng thiết bị mục tiêu mà không làm mất nhịp sống hoặc lệnh. Về độ tin cậy, lệnh phải có mã định danh để chống thực thi trùng, có thời hạn và có trạng thái kết thúc rõ ràng. Triển khai mô hình phải theo nguyên tắc kích hoạt nguyên tử: phiên bản mới chỉ trở thành phiên bản hoạt động sau khi tải đủ, kiểm tra hàm băm, nạp thử và vượt qua health check.

Về an toàn thông tin, mọi Edge Agent cần có danh tính riêng; kết nối phải được mã hóa; API phải kiểm soát quyền theo vai trò; bí mật không được ghi trực tiếp vào mã nguồn hoặc tệp log. Lệnh từ AI Agent phải đi qua cùng lớp xác thực và chính sách như lệnh do người dùng tạo. Những thao tác có ảnh hưởng cao như triển khai diện rộng, dừng dịch vụ hoặc khôi phục hàng loạt phải yêu cầu phê duyệt rõ ràng.

Về khả năng quan sát, hệ thống phải cung cấp log, metric và dấu vết của thao tác phân tán. Mốc thời gian cần được đồng bộ để có thể đối chiếu sự kiện giữa control plane và Edge Agent. Về khả năng bảo trì, giao thức và gói mô hình phải có phiên bản; các thành phần có giao diện tách biệt để có thể thay thế cơ sở dữ liệu, message broker hoặc bộ thực thi mô hình mà không làm thay đổi toàn bộ hệ thống.

1.6 Các hướng tiếp cận hiện có và khoảng trống

Hướng thứ nhất là tối ưu riêng mô hình rồi chép thủ công sang thiết bị. Cách làm này phù hợp với thử nghiệm nhỏ nhưng khó kiểm soát phiên bản, khó lặp lại và không cho biết thiết bị nào đang chạy cấu hình nào. Hướng thứ hai sử dụng công cụ quản lý thiết bị hoặc container tổng quát. Hướng này giải quyết triển khai phần mềm nhưng thường chưa mô tả đầy đủ quan hệ giữa mô hình, cấu hình tiền xử lý, bộ thực thi và chỉ số chất lượng nhận diện. Hướng thứ ba sử dụng nền tảng MLOps từ đám mây. Nền tảng loại này có nhiều chức năng nhưng có thể tạo phụ thuộc nhà cung cấp, chi phí vận hành hoặc yêu cầu kết nối không phù hợp với môi trường biên.

Đề án lựa chọn kiến trúc nhẹ, tách control plane khỏi Edge Agent và coi gói mô hình là một hiện vật có phiên bản. Điểm khác biệt quan trọng là kết nối giữa quy trình tối ưu với quản lý vòng đời triển khai. Một phiên bản chỉ được phát hành khi có manifest, hàm băm, thông tin tương thích và kết quả đánh giá. AI Agent không kết nối trực tiếp tới thiết bị mà thao tác qua API quản lý, nhờ đó thừa hưởng kiểm soát quyền, kiểm toán và cơ chế phê duyệt.

1.7 Đóng góp của đề án

Đề án hướng tới bốn đóng góp chính. Thứ nhất, đề án xây dựng quy trình tối ưu YOLO có đường chuẩn và tiêu chí so sánh rõ ràng, tránh đánh giá chỉ dựa trên kích thước tệp hoặc FPS. Thứ hai, đề án đề xuất kiến trúc quản lý Edge Agent hỗ trợ danh tính thiết bị, phân phối mô hình, telemetry, lệnh có trạng thái và khôi

phục. Thứ ba, đồ án xây dựng Dashboard hợp nhất dữ liệu kỹ thuật thành góc nhìn phục vụ vận hành. Thứ tư, đồ án tích hợp AI Agent theo nguyên tắc công cụ giới hạn quyền, lập kế hoạch trước khi thực thi và bắt buộc phê duyệt cho hành động rủi ro.

1.8 Kết chương

Chương I đã chuyển yêu cầu tổng quát của đề tài thành bài toán có đầu vào, đầu ra, mục tiêu tối ưu và tập yêu cầu có thể kiểm thử. Kết quả phân tích cho thấy tối ưu mô hình và quản lý thiết bị phải được thiết kế như hai phần của cùng một vòng đời. Chương II tiếp tục trình bày các cơ sở lý thuyết và công nghệ cần thiết để lựa chọn phương pháp tối ưu, giao thức quản lý và cơ chế điều khiển bằng AI Agent.

CHƯƠNG II. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

Chương I đã xác định hệ thống cần đồng thời bảo đảm chất lượng nhận diện, hiệu năng thiết bị và khả năng vận hành. Chương này trình bày những khái niệm làm cơ sở cho thiết kế: mô hình YOLO và phép đo chất lượng, các kỹ thuật tối ưu suy luận, đặc trưng của Edge AI, quản lý đội thiết bị phân tán, khả năng quan sát và mô hình AI Agent sử dụng công cụ.

2.1 Mô hình phát hiện đối tượng YOLO

Phát hiện đối tượng yêu cầu xác định đồng thời lớp và hộp bao của từng đối tượng trong ảnh. Các phiên bản YOLO đầu tiên đặt bài toán dưới dạng hồi quy trực tiếp từ ảnh sang tập hộp bao và xác suất lớp, nhờ đó có thể xử lý toàn ảnh trong một mạng duy nhất [1], [2]. Các thế hệ sau tiếp tục cải tiến backbone, neck, hàm mất mát, chiến lược gán nhãn và hậu xử lý; YOLOv4 là một ví dụ về việc phối hợp nhiều kỹ thuật để cân bằng tốc độ với độ chính xác [3]. Đồ án không giả định rằng phiên bản lớn nhất luôn phù hợp nhất, mà lựa chọn biến thể dựa trên ràng buộc của thiết bị và dữ liệu mục tiêu.

Chuỗi suy luận thường gồm bốn bước. Ảnh đầu vào được đọc và biến đổi về kích thước yêu cầu; tensor được chuẩn hóa và chuyển sang bố trí bộ nhớ phù hợp; mạng nơ-ron sinh các dự đoán thô; cuối cùng hệ thống giải mã hộp, lọc theo ngưỡng và loại bỏ các hộp trùng lặp. Khi đánh giá trên Edge AI, thời gian cần đo phải bao gồm cả các bước cần thiết của ứng dụng, không chỉ thời gian chạy phần mạng. Nếu tiền xử lý nằm trên CPU trong khi mạng chạy trên NPU, chi phí sao chép bộ nhớ có thể trở thành nút thắt.

2.1.1 Giao trên hợp và hậu xử lý

Với hộp dự đoán B_p và hộp chuẩn B_g , chỉ số giao trên hợp được tính theo:

$$\text{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \quad (2.1)$$

Một dự đoán được xem là đúng khi lớp phù hợp và IoU vượt ngưỡng đánh giá. Trong hậu xử lý, non-maximum suppression giữ hộp có điểm tin cậy cao và loại các hộp cùng lớp có mức chồng lấn lớn. Ngưỡng tin cậy và ngưỡng IoU ảnh hưởng trực tiếp đến cân bằng giữa bỏ sót và cảnh báo sai, do đó phải được lưu cùng phiên

bản cấu hình.

2.1.2 Chỉ số chất lượng nhận diện

Precision và recall được xác định bởi:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad (2.2)$$

trong đó TP , FP và FN lần lượt là dự đoán đúng, dự đoán sai và đối tượng bị bỏ sót. Average Precision là diện tích dưới đường precision–recall của một lớp; mAP là trung bình AP trên các lớp và, tùy quy ước, trên nhiều ngưỡng IoU. Khi báo cáo mAP phải ghi rõ quy ước, chẳng hạn mAP tại IoU 0,5 hay mAP trung bình từ 0,5 đến 0,95. Ngoài giá trị tổng, đồ án cần quan sát AP theo lớp và recall của vật thể nhỏ để tránh che khuất lỗi bởi một số lớp dễ.

2.2 Tối ưu mô hình cho thiết bị biên

Tối ưu cần bắt đầu từ đo lường. Mô hình cơ sở được cố định phiên bản, dữ liệu đánh giá và cấu hình đầu vào. Sau đó mỗi thay đổi được xem như một thí nghiệm có tham số và kết quả. Các nhóm kỹ thuật chính được tóm tắt trong Bảng 2.1.

Bảng 2.1: Các nhóm kỹ thuật tối ưu mô hình

Kỹ thuật	Lợi ích kỳ vọng	Rủi ro hoặc điều kiện
Giảm kích thước đầu vào	Giảm số phép tính và bộ nhớ trung gian	Có thể giảm khả năng nhận diện vật thể nhỏ
Chọn biến thể nhỏ hơn	Giảm tham số, độ trễ và công suất	Khả năng biểu diễn thấp hơn
Lượng tử hóa	Giảm kích thước và tăng tốc trên phần cứng hỗ trợ số nguyên	Cần dữ liệu hiệu chỉnh; một số lớp nhạy với sai số
Cắt tỉa	Loại trọng số hoặc kênh ít quan trọng	Tăng tốc thực tế phụ thuộc cấu trúc và bộ thực thi
Chưng cất tri thức	Mô hình nhỏ học từ mô hình giáo viên	Phải huấn luyện lại và lựa chọn hàm mất mát phù hợp
Hợp nhất và biên dịch đồ thị	Giảm overhead, chọn kernel tối ưu	Phụ thuộc phép toán được backend hỗ trợ
Tối ưu tiên/hậu xử lý	Giảm sao chép và xử lý tuần tự trên CPU	Cần kiểm chứng đầu ra tương đương

2.2.1 Lượng tử hóa

Lượng tử hóa ánh xạ trọng số và/hoặc kích hoạt từ biểu diễn dấu phẩy động sang miền có độ rộng bit thấp hơn. Với lượng tử hóa tuyến tính, giá trị thực r được xấp xỉ bởi số nguyên q thông qua hệ số tỷ lệ s và điểm không z :

$$q = \text{clip} \left(\text{round} \left(\frac{r}{s} \right) + z \right). \quad (2.3)$$

PTQ thực hiện lượng tử hóa sau huấn luyện và cần một tập hiệu chỉnh đại diện. QAT mô phỏng sai số lượng tử trong huấn luyện, thường giữ chất lượng tốt hơn nhưng làm tăng chi phí thực hiện. Nghiên cứu của Jacob và cộng sự mô tả cách suy luận số nguyên có thể được triển khai hiệu quả trên phần cứng hạn chế [4]. Trong đồ án, mô hình lượng tử chỉ được chấp nhận khi cả chất lượng và tốc độ đầu cuối đạt tiêu chí; giảm kích thước tệp mà không tăng tốc không được xem là đủ.

2.2.2 Cắt tỉa và chưng cất tri thức

Cắt tỉa loại bỏ trọng số, kênh hoặc khối có mức đóng góp thấp. Cắt tỉa không cấu trúc có thể tạo nhiều số không nhưng không luôn tăng tốc trên bộ thực thi phổ thông. Cắt tỉa có cấu trúc dễ chuyển thành giảm kích thước tensor hơn, song có thể làm giảm độ chính xác mạnh nếu tỷ lệ quá lớn. Deep Compression cho thấy hiệu quả của việc kết hợp cắt tỉa, lượng tử hóa và mã hóa, đồng thời nhấn mạnh nhu cầu tinh chỉnh sau nén [5].

Chưng cất tri thức sử dụng đầu ra hoặc đặc trưng của mô hình lớn làm tín hiệu huấn luyện cho mô hình nhỏ. Mục tiêu có thể kết hợp mất mát nhân thật và mất mát mô phỏng giáo viên. Phương pháp này giúp mô hình nhỏ duy trì một phần khả năng biểu diễn của mô hình lớn [6]. Đối với phát hiện đối tượng, cần xác định rõ thành phần được chưng cất như logit lớp, đặc trưng trung gian hoặc dự đoán hộp.

2.2.3 Biên dịch đồ thị và bộ thực thi

ONNX cung cấp biểu diễn trung gian giúp trao đổi mô hình giữa framework huấn luyện và công cụ triển khai [7]. Bộ biên dịch hoặc runtime có thể thực hiện constant folding, hợp nhất phép toán, lựa chọn kernel và tối ưu bố trí bộ nhớ. Trên nền tảng NVIDIA, TensorRT là một ví dụ về bộ tối ưu và runtime cho suy luận hiệu năng cao [8]. Các nền tảng khác có bộ công cụ riêng cho NPU hoặc GPU tích hợp. Vì kết quả chuyển đổi có thể sai khác số học, đồ án cần kiểm tra đầu ra theo các mẫu cố định trước khi đo hiệu năng.

2.3 Đặc trưng của Edge AI

Edge computing đưa năng lực tính toán và lưu trữ tới gần nguồn dữ liệu nhằm giảm độ trễ và lưu lượng về trung tâm [9], [10]. Thiết bị biên không đồng nhất về kiến trúc CPU, bộ tăng tốc, dung lượng RAM và hệ điều hành. Chúng cũng có thể hoạt động trong điều kiện nhiệt độ, nguồn điện và mạng thay đổi. Do đó, một cấu hình đạt FPS cao trong phép đo ngắn chưa chắc duy trì được lâu dài khi thiết bị giảm xung do nhiệt.

Một phép đánh giá phù hợp phải có giai đoạn khởi động, số vòng lặp đủ lớn và báo cáo phân phối độ trễ. Độ trễ L nên được tách thành:

$$L = L_{capture} + L_{pre} + L_{infer} + L_{post} + L_{publish}, \quad (2.4)$$

trong đó các thành phần lần lượt là thu nhận, tiền xử lý, suy luận, hậu xử lý và công bố kết quả. Thông lượng có thể được đo bằng FPS trung bình, nhưng với luồng nối tiếp, FPS không nên được suy ra máy móc từ nghịch đảo độ trễ nếu còn hàng đợi hoặc xử lý song song.

2.4 Quản lý Edge Agent và vòng đời mô hình

Edge Agent là dịch vụ chạy trên thiết bị, đóng vai trò cầu nối giữa bộ thực thi suy luận với control plane. Agent duy trì danh tính, gửi heartbeat, thu telemetry, nhận desired state và hội tụ thiết bị về trạng thái mong muốn. Mô hình desired state phù hợp hơn chuỗi lệnh rời rạc trong trường hợp mất mạng: khi kết nối trở lại, Agent có thể so sánh phiên bản mong muốn với phiên bản hiện tại rồi tiếp tục quá trình đồng bộ.

Gói mô hình cần bao gồm tệp nhị phân và manifest. Manifest mô tả mã phiên bản, hàm băm, backend, kiểu dữ liệu, kích thước đầu vào, danh sách lớp, ngưỡng mặc định và yêu cầu tối thiểu của thiết bị. Control plane lưu metadata, còn tệp lớn có thể nằm trong kho hiện vật. Trước khi kích hoạt, Edge Agent kiểm tra dung lượng, tính tương thích và hàm băm. Bản đang chạy được giữ lại đến khi phiên bản mới vượt qua health check, tạo cơ sở cho rollback.

MQTT là giao thức publish/subscribe nhẹ, hỗ trợ nhiều mức chất lượng dịch vụ và phù hợp với telemetry hoặc lệnh trong môi trường thiết bị [11]. REST phù hợp với thao tác quản trị và truy vấn tài nguyên; WebSocket hoặc Server-Sent Events phù hợp với cập nhật Dashboard gần thời gian thực. Đồ án có thể kết hợp các giao thức nhưng phải xác định một nguồn sự thật duy nhất cho trạng thái và quy tắc chống xử lý trùng.

2.5 Dashboard, telemetry và cảnh báo

Khả năng quan sát gồm ba nhóm dữ liệu: metric định lượng, log sự kiện và dấu vết của luồng thao tác. Metric của thiết bị bao gồm nhiệt độ, bộ nhớ, mức sử dụng bộ xử lý, FPS và độ trễ. Metric nghiệp vụ có thể gồm số đối tượng, tỷ lệ khung hình bị bỏ và số cảnh báo. Log cần có mức độ, mốc thời gian, mã thiết bị, phiên bản dịch vụ và mã tương quan. Dấu vết triển khai nổi yêu cầu của người dùng với lệnh, phản hồi của Agent và thay đổi trạng thái.

Dashboard không nên chỉ trình bày nhiều biểu đồ. Màn hình tổng quan cần trả lời nhanh ba câu hỏi: hệ thống có bình thường hay không, thành phần nào cần

chú ý và hành động tiếp theo là gì. Màu sắc cảnh báo phải đi kèm trạng thái văn bản, tránh phụ thuộc hoàn toàn vào màu. Dữ liệu lịch sử phải cho phép so sánh trước và sau triển khai để phát hiện suy giảm hiệu năng.

2.6 AI Agent sử dụng công cụ có kiểm soát

Trong phạm vi đề án, AI Agent là lớp giao tiếp và lập kế hoạch dựa trên mô hình ngôn ngữ. Agent không truy cập trực tiếp cơ sở dữ liệu hoặc thiết bị, mà gọi các công cụ có hợp đồng đầu vào–đầu ra rõ ràng, ví dụ lấy trạng thái thiết bị, truy vấn metric, tạo yêu cầu triển khai hoặc khôi phục. Quy trình gồm nhận mục tiêu, thu thập ngữ cảnh, tạo kế hoạch, kiểm tra chính sách, xin phê duyệt nếu cần, thực thi từng bước và tóm tắt kết quả.

Rủi ro chính của AI Agent là diễn giải sai ý định, tạo tham số không hợp lệ, vượt quyền hoặc bị nội dung bên ngoài thao túng. Biện pháp kiểm soát gồm danh sách công cụ cho phép, schema chặt chẽ, RBAC, giới hạn phạm vi thiết bị, chế độ chỉ đọc mặc định, xác nhận đối với thao tác ghi, idempotency key và audit log. Kiến trúc zero trust khuyến nghị không mặc nhiên tin cậy chỉ dựa trên vị trí mạng; mỗi yêu cầu phải được xác thực và ủy quyền theo ngữ cảnh [12]. Nguyên tắc này cũng được áp dụng cho mọi lệnh do AI Agent khởi tạo.

2.7 Kết chương

Chương II đã trình bày cơ sở để đánh giá mô hình YOLO, các kỹ thuật giảm chi phí suy luận và những yêu cầu đặc thù của Edge AI. Chương cũng làm rõ vai trò của manifest, desired state, telemetry, Dashboard và AI Agent có công cụ giới hạn quyền. Những khái niệm này được cụ thể hóa thành kiến trúc và luồng xử lý trong Chương III.

CHƯƠNG III. THIẾT KẾ GIẢI PHÁP ĐỀ XUẤT

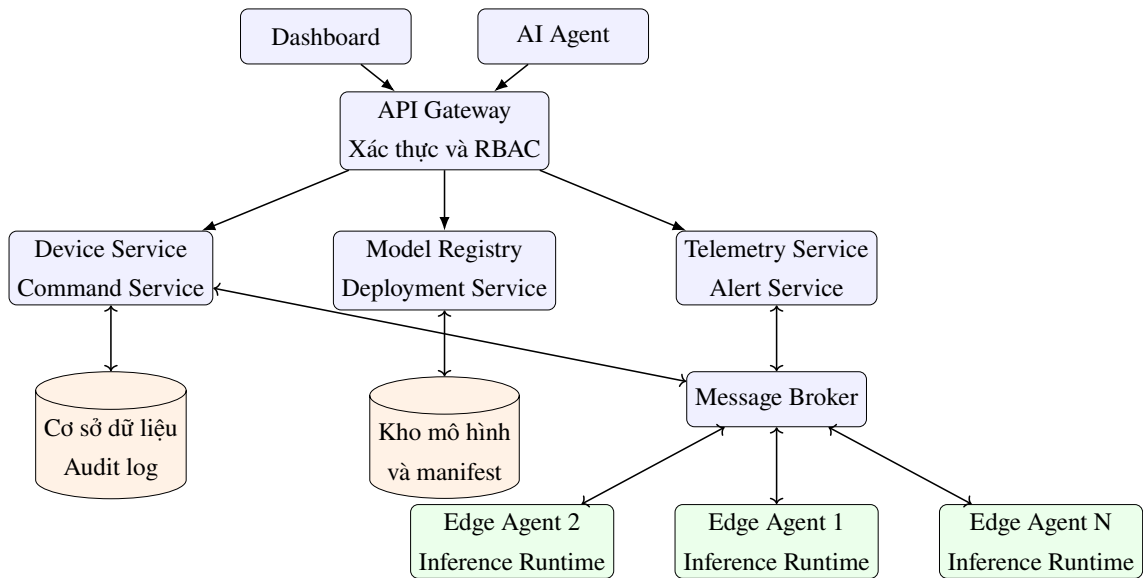
Trên cơ sở yêu cầu của Chương I và nền tảng của Chương II, chương này đề xuất kiến trúc tổng thể cho giải pháp. Thiết kế tách rõ mặt phẳng quản lý tập trung với mặt phẳng thực thi tại biên, đồng thời đặt AI Agent sau lớp công cụ và chính sách. Mọi gói mô hình, cấu hình và lệnh đều có phiên bản, có thể kiểm tra và truy vết.

3.1 Nguyên tắc thiết kế

Giải pháp tuân theo sáu nguyên tắc. Thứ nhất, thiết bị tiếp tục suy luận trong khoảng thời gian mất kết nối nếu không có lỗi nghiêm trọng. Thứ hai, control plane lưu trạng thái mong muốn, còn Edge Agent chịu trách nhiệm hội tụ an toàn về trạng thái đó. Thứ ba, mô hình và cấu hình được đóng gói như hiện vật bất biến; thay đổi tạo phiên bản mới thay vì sửa âm thầm. Thứ tư, mọi lệnh thay đổi trạng thái đều được xác thực, kiểm tra quyền và ghi nhật ký. Thứ năm, bản cập nhật phải có health check và rollback. Thứ sáu, AI Agent chỉ sử dụng công cụ được công bố và không có đường tắt tới thiết bị.

3.2 Kiến trúc tổng thể

Hình 3.1 mô tả các thành phần chính. Người dùng thao tác qua Dashboard hoặc hội thoại với AI Agent. Cả hai kênh đi qua API Gateway và dịch vụ định danh. Control plane gồm Device Service, Model Registry, Deployment Service, Telemetry Service và Command Service. Message Broker chuyển heartbeat, telemetry và lệnh gần thời gian thực. Kho dữ liệu lưu metadata, chuỗi thời gian, audit log và tệp mô hình. Ở phía biên, Edge Agent quản lý bộ thực thi suy luận, nguồn dữ liệu và cơ chế cập nhật.



Hình 3.1: Kiến trúc tổng thể của giải pháp đề xuất

3.2.1 Control plane

Device Service lưu hồ sơ thiết bị, năng lực phần cứng, nhãn và trạng thái kết nối. Model Registry quản lý metadata của gói mô hình; tệp nhị phân được lưu trong kho hiện vật và tham chiếu bằng URI có thời hạn. Deployment Service chuyển yêu cầu phát hành thành desired state cho từng thiết bị, theo dõi tiến độ và áp dụng chiến lược theo đợt. Command Service quản lý lệnh ngắn hạn như khởi động lại dịch vụ hoặc thu thập chẩn đoán. Telemetry Service tiếp nhận, chuẩn hóa và lưu chuỗi thời gian. Alert Service đánh giá luật cảnh báo và tránh gửi lặp bằng cơ chế nhóm sự kiện.

3.2.2 Edge plane

Edge Agent gồm các mô-đun: Identity, Communication, Model Manager, Configuration Manager, Inference Supervisor, Telemetry Collector và Local Store. Identity quản lý chứng thư hoặc token thiết bị. Communication duy trì kết nối, backoff khi lỗi và hàng đợi cục bộ. Model Manager tải gói, kiểm tra hàm băm, giải nén, kiểm tra tương thích và đổi liên kết phiên bản đang hoạt động. Inference Supervisor khởi chạy bộ thực thi, kiểm tra sức khỏe và khởi động lại theo chính sách. Local Store giữ phiên bản ổn định gần nhất, cấu hình cuối cùng và các sự kiện chưa gửi.

3.3 Mô hình dữ liệu

Các thực thể chính được trình bày trong Bảng 3.1. Trường phiên bản và mốc thời gian được sử dụng để xử lý cập nhật đồng thời. Mọi thay đổi trạng thái quan trọng đều sinh Audit Event bất biến.

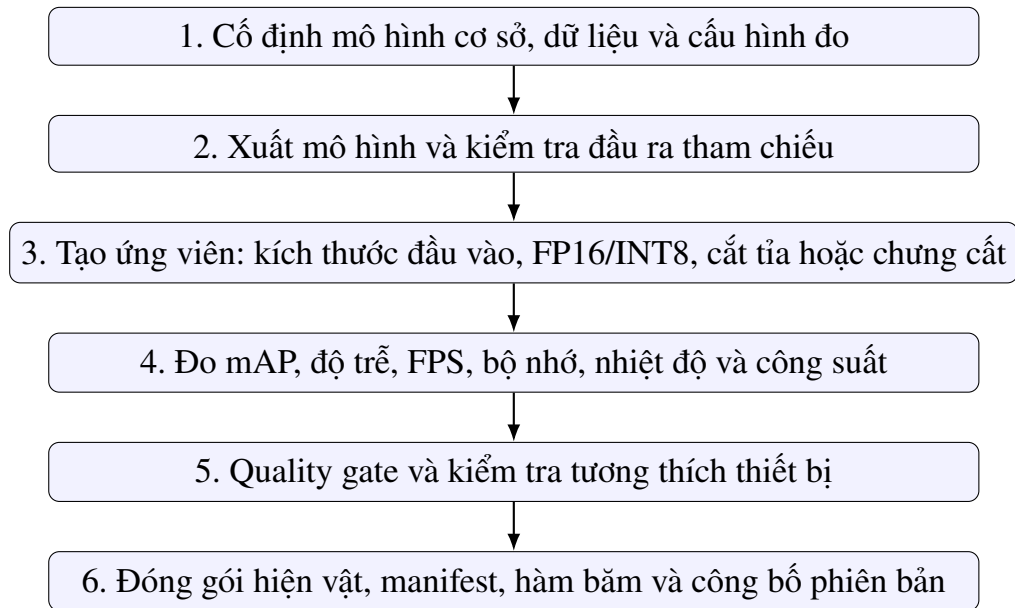
Bảng 3.1: Các thực thể dữ liệu chính

Thực thể	Vai trò	Thuộc tính tiêu biểu
Device	Đại diện Edge Agent	ID, nhãn, năng lực, phiên bản Agent, last seen, trạng thái
Model Artifact	Gói mô hình bất biến	ID, phiên bản, URI, SHA-256, backend, precision, input shape
Deployment	Yêu cầu triển khai	phạm vi, phiên bản đích, chiến lược, người tạo, trạng thái
Desired State	Trạng thái mong muốn	model version, config version, generation
Reported State	Trạng thái thực tế	phiên bản hiện tại, health, lỗi, thời điểm báo cáo
Command	Lệnh có vòng đời	command ID, loại, tham số, hạn dùng, trạng thái, kết quả
Telemetry	Mẫu đo theo thời gian	device ID, metric, value, timestamp, tags
Audit Event	Bản ghi kiểm toán	actor, action, resource, before/after, correlation ID

3.4 Quy trình tối ưu và đóng gói mô hình

Quy trình tối ưu trong Hình 3.2 bắt đầu bằng mô hình cơ sở đã đóng băng và bộ dữ liệu đánh giá cố định. Sau phép đo đường chuẩn, các ứng viên được tạo bằng thay đổi kích thước đầu vào, biến thể mô hình, precision hoặc kỹ thuật lượng

tử hóa. Mỗi ứng viên trải qua kiểm tra tương đương đầu ra, đánh giá độ chính xác, đo hiệu năng dài hạn và kiểm tra tương thích. Chỉ ứng viên đạt ngưỡng mới được đóng gói, ký manifest và công bố vào Model Registry.



Hình 3.2: Quy trình tối ưu và phát hành mô hình

Manifest tối thiểu chứa tên mô hình, phiên bản ngữ nghĩa, hàm băm SHA-256, backend, precision, kích thước tensor, danh sách lớp, ngưỡng mặc định, phiên bản runtime tối thiểu và chỉ số đánh giá. Manifest không chứa khóa truy cập. Cùng một tệp mô hình nhưng khác cấu hình tiền xử lý hoặc danh sách lớp phải được xem là gói khác phiên bản, vì đầu ra ứng dụng có thể thay đổi.

3.5 Vòng đời thiết bị và triển khai

Thiết bị chuyển qua các trạng thái chưa đăng ký, đang đăng ký, trực tuyến, suy giảm, ngoại tuyến và bị vô hiệu hóa. Trạng thái trực tuyến chỉ được xác nhận khi heartbeat còn hiệu lực và dịch vụ suy luận vượt qua health check. Trạng thái suy giảm được dùng khi Agent vẫn kết nối nhưng một metric vượt ngưỡng hoặc bộ thực thi gặp lỗi lặp lại.

Quá trình triển khai gồm bảy bước: (i) control plane kiểm tra quyền và tính tương thích sơ bộ; (ii) tạo Deployment và tăng generation của desired state; (iii) Edge Agent nhận desired state; (iv) Agent tải gói vào vùng tạm; (v) kiểm tra hàm băm, dung lượng và khả năng nạp; (vi) chuyển sang phiên bản mới rồi chạy health check; (vii) báo cáo thành công hoặc rollback. Khi triển khai cho nhiều thiết bị, hệ thống thực hiện theo lô nhỏ trước, quan sát trong khoảng thời gian xác định rồi mới

mở rộng. Nếu tỷ lệ lỗi vượt ngưỡng, đợt triển khai tự động dừng.

3.6 Giao thức lệnh và chống thực thi trùng

Mỗi lệnh có `command_id` duy nhất, loại lệnh, tham số, mốc tạo, thời hạn, người khởi tạo và idempotency key. Agent lưu ngăn hạn các mã đã xử lý; nếu nhận lại cùng mã, Agent trả kết quả trước đó thay vì thực thi lần hai. Vòng đời lệnh gồm *pending*, *delivered*, *accepted*, *running*, *succeeded*, *failed*, *expired* và *cancelled*. Control plane không suy đoán thành công chỉ dựa trên việc message broker đã chuyển thông điệp.

Các lệnh được chia ba mức rủi ro. Lệnh đọc như lấy trạng thái có thể thực hiện ngay. Lệnh thay đổi cục bộ có khả năng khôi phục, ví dụ cập nhật ngưỡng cho một thiết bị, yêu cầu quyền vận hành và ghi audit. Lệnh ảnh hưởng cao như dừng nhiều thiết bị hoặc triển khai diện rộng yêu cầu quyền quản trị và một bước phê duyệt độc lập.

3.7 Thiết kế Dashboard

Dashboard gồm năm nhóm màn hình. Trang tổng quan hiển thị số thiết bị theo trạng thái, cảnh báo đang mở, phân bố phiên bản mô hình và xu hướng độ trễ. Trang danh sách thiết bị hỗ trợ lọc theo nhãn, phần cứng, trạng thái và phiên bản. Trang chi tiết thiết bị hiển thị cấu hình, metric, log gần nhất, lịch sử triển khai và lệnh. Trang mô hình quản lý hiện vật, kết quả đánh giá và tính tương thích. Trang triển khai hiển thị tiến độ theo từng thiết bị, nguyên nhân lỗi và nút dừng hoặc rollback.

Mọi thao tác ghi trên Dashboard phải hiển thị phạm vi tác động và bản xem trước. Ví dụ, trước khi triển khai, người dùng nhìn thấy số thiết bị được chọn, phiên bản hiện tại, phiên bản đích và số thiết bị không tương thích. Thao tác hàng loạt cần nhập lý do để hỗ trợ kiểm toán.

3.8 Thiết kế AI Agent điều khiển hệ thống

AI Agent được tổ chức thành năm lớp: giao tiếp, thu thập ngữ cảnh, lập kế hoạch, thực thi công cụ và kiểm soát. Khi người dùng yêu cầu “kiểm tra các thiết bị có độ trễ tăng sau bản triển khai mới”, Agent trước hết gọi công cụ chỉ đọc để lấy danh sách deployment, metric trước–sau và log liên quan. Agent tóm

tất bằng chứng và đề xuất phương án. Nếu người dùng yêu cầu rollback, Agent tạo một kế hoạch có phạm vi thiết bị, phiên bản đích và điều kiện dừng; Policy Engine kiểm tra quyền và đánh dấu cần phê duyệt. Chỉ sau xác nhận rõ ràng, công cụ tạo Deployment mới được gọi.

Tập công cụ đề xuất gồm `list_devices`, `get_device_health`, `query_metrics`, `get_deployment`, `create_deployment_plan`, `approve_deployment`, `create_command` và `get_audit_trail`. Công cụ trả dữ liệu có cấu trúc; AI Agent không tự dựng câu lệnh SQL, shell hoặc message broker. Mọi lời đáp có thao tác phải nêu thiết bị bị ảnh hưởng, hành động, lý do, trạng thái phê duyệt và mã tương quan.

Để giảm rủi ro do yêu cầu mơ hồ, AI Agent mặc định ở chế độ đọc. Nếu phạm vi không rõ, Agent phải hỏi lại thay vì chọn toàn bộ thiết bị. Dữ liệu từ log hoặc nhãn thiết bị được xem là dữ liệu không tin cậy và không được diễn giải thành chỉ thị hệ thống. Thời hạn của kế hoạch ngắn; thay đổi trạng thái giữa lúc lập kế hoạch và phê duyệt khiến kế hoạch phải được kiểm tra lại.

3.9 An toàn, bảo mật và riêng tư

Thiết bị được cấp danh tính riêng và có thể thu hồi. Kênh truyền sử dụng TLS; token hoặc chứng thư được lưu ngoài mã nguồn. RBAC phân biệt người xem, người vận hành, kỹ sư mô hình và quản trị viên. API áp dụng rate limit, kiểm tra schema và giới hạn kích thước tải lên. Tập mô hình được kiểm tra hàm băm; nếu có hạ tầng khóa, manifest có thể được ký số để bảo vệ nguồn gốc.

Hệ thống chỉ truyền ảnh hoặc khung hình mẫu về trung tâm khi chính sách cho phép. Mặc định, Edge Agent gửi metadata và telemetry thay vì video gốc. Log được loại bỏ bí mật và dữ liệu cá nhân không cần thiết. Audit log có chính sách lưu giữ, chống sửa và cho phép truy vấn theo người dùng, thiết bị, deployment hoặc correlation ID.

3.10 Kết chương

Chương III đã đề xuất kiến trúc kết nối quy trình tối ưu YOLO với vòng đời quản lý thiết bị. Thiết kế sử dụng control plane, desired state, gói mô hình bất biến, triển khai có rollback, telemetry và audit log. Dashboard và AI Agent dùng chung API và chính sách quyền, nhờ đó điều khiển bằng ngôn ngữ tự nhiên không làm suy yếu cơ chế an toàn. Chương IV trình bày cách ánh xạ thiết kế này thành các

mô-đun phần mềm và quy trình triển khai cụ thể.

CHƯƠNG IV. CÀI ĐẶT VÀ TRIỂN KHAI HỆ THỐNG

Chương III đã xác định các thành phần và giao thức của giải pháp. Chương này mô tả cách tổ chức mã nguồn, hiện thực Edge Agent, control plane, Dashboard và AI Agent, đồng thời trình bày quy trình triển khai. Vì công nghệ cụ thể cần khớp với mã nguồn thực tế của sinh viên, các vị trí liên quan đến tên framework và phiên bản được đánh dấu để cập nhật trước khi nộp.

4.1 Môi trường và tổ chức mã nguồn

Hệ thống nên được tổ chức theo các mô-đun có thể phát triển và kiểm thử độc lập. Mô-đun *edge-agent* chứa giao tiếp thiết bị, quản lý mô hình và giám sát tiến trình suy luận. Mô-đun *control-plane* chứa API, nghiệp vụ thiết bị, triển khai, lệnh và cảnh báo. Mô-đun *dashboard* chứa giao diện web. Mô-đun *ai-agent* chứa định nghĩa công cụ, policy hook và luồng phê duyệt. Thư mục *contracts* lưu schema message và API dùng chung; thư mục *deployment* lưu cấu hình triển khai; thư mục *tests* chứa kiểm thử tích hợp và kịch bản mô phỏng thiết bị.

Bảng 4.1: Công nghệ sử dụng trong phiên bản cài đặt

Thành phần	Vai trò	Công nghệ/phiên bản thực tế
Edge Agent	Quản lý suy luận và giao tiếp	[CẦN BỔ SUNG: ngôn ngữ, framework]
Inference Runtime	Chạy mô hình tối ưu	[CẦN BỔ SUNG: runtime/SDK]
Backend API	Nghiệp vụ control plane	[CẦN BỔ SUNG: framework backend]
Message Broker	Heartbeat, telemetry, lệnh	[CẦN BỔ SUNG: broker]
Cơ sở dữ liệu	Metadata và audit log	[CẦN BỔ SUNG: hệ quản trị CSDL]
Kho chuỗi thời gian	Metric theo thời gian	[CẦN BỔ SUNG: nếu có]
Dashboard	Giao diện quản trị	[CẦN BỔ SUNG: framework frontend]
AI Agent	Hội thoại và gọi công cụ	[CẦN BỔ SUNG: mô hình/nền tảng]
Đóng gói	Triển khai dịch vụ	[CẦN BỔ SUNG: container/service manager]

Các phiên bản phụ thuộc cần được khóa trong tệp cấu hình để bản dựng có thể lặp lại. Bí mật được truyền qua biến môi trường hoặc kho bí mật, không lưu trong repository. Mỗi dịch vụ ghi số phiên bản và commit tại thời điểm khởi động, giúp đối chiếu log với mã nguồn.

4.2 Hiện thực Edge Agent

Khi khởi động, Edge Agent đọc cấu hình bootstrap tối thiểu gồm địa chỉ control plane, thông tin định danh và thư mục dữ liệu. Agent xác thực với máy chủ,

lấy desired state, so sánh generation với trạng thái cục bộ rồi mới quyết định tải mô hình hoặc cập nhật cấu hình. Nếu máy chủ không truy cập được, Agent dùng cấu hình tốt gần nhất và tiếp tục suy luận trong chế độ ngoại tuyến.

4.2.1 Quản lý tiến trình suy luận

Inference Supervisor tách khỏi luồng giao tiếp để sự cố của camera hoặc runtime không làm mất khả năng quản trị. Supervisor khởi chạy worker, kiểm tra heartbeat nội bộ và áp dụng backoff khi worker lỗi liên tiếp. Trạng thái sức khỏe gồm ba mức: *healthy* khi luồng suy luận và xuất kết quả bình thường; *degraded* khi FPS dưới ngưỡng, hàng đợi tăng hoặc nhiệt độ cao; *unhealthy* khi runtime không nạp được mô hình hoặc nguồn dữ liệu gián đoạn quá thời hạn.

Chuỗi xử lý hình ảnh sử dụng hàng đợi có kích thước hữu hạn. Khi tốc độ camera lớn hơn khả năng suy luận, chính sách bỏ khung hình cũ phù hợp hơn để tránh độ trễ tích lũy. Mỗi kết quả gắn mốc thời gian thu nhận, mốc bắt đầu suy luận, mốc hoàn thành và phiên bản mô hình. Nhờ đó, hệ thống phân biệt được độ trễ mô hình với độ trễ do hàng đợi.

4.2.2 Cập nhật mô hình nguyên tử

Model Manager tải tệp vào thư mục tạm, đối chiếu dung lượng và SHA-256 với manifest. Sau đó, Agent tạo runtime thử nghiệm, nạp mô hình và chạy một tập đầu vào kiểm tra. Nếu bước này thành công, phiên bản mới được đặt vào thư mục bất biến và con trỏ *current* được chuyển bằng thao tác nguyên tử. Phiên bản trước chưa bị xóa ngay mà được giữ theo chính sách dung lượng.

Thuật toán 4.1 mô tả luồng cập nhật. Phần kiểm tra tương thích phải chạy trước khi dừng worker hiện tại, nhờ đó giảm thời gian gián đoạn.

Listing 4.1: Giả mã cập nhật mô hình tại Edge Agent

```
function apply_model(desired):
    if desired.version == reported.version:
        return ALREADY_CONVERGED
    package = download_to_staging(desired.uri)
    verify_sha256(package, desired.sha256)
    verify_compatibility(package.manifest, device.
        capabilities)
    candidate = create_runtime(package)
    candidate.warmup(health_samples)
```

```
if not candidate.health_check():
    return FAILED_WITHOUT_SWITCH
previous = inference_supervisor.current_runtime
inference_supervisor.atomic_switch(candidate)
if not observe_health(candidate, rollout_window):
    inference_supervisor.atomic_switch(previous)
    return ROLLED_BACK
persist_reported_state(desired.version)
return SUCCEEDED
```

4.2.3 Telemetry và bộ đệm ngoại tuyến

Telemetry Collector lấy metric hệ thống theo chu kỳ và metric suy luận theo cửa sổ. Metric tần suất cao được tổng hợp tại thiết bị thành trung bình, min, max và các phân vị trước khi gửi để giảm băng thông. Khi mất kết nối, Agent lưu sự kiện quan trọng và một lượng metric giới hạn theo dung lượng. Sau khi kết nối lại, dữ liệu được gửi theo lô; mỗi lô có sequence number để máy chủ phát hiện thiếu hoặc trùng.

4.3 Hiện thực control plane

Backend API phân tách lớp giao tiếp, lớp nghiệp vụ và lớp truy cập dữ liệu. Controller chỉ kiểm tra schema và chuyển ngữ cảnh xác thực; service thực hiện quy tắc như kiểm tra tính tương thích, tạo deployment hoặc cập nhật trạng thái; repository quản lý lưu trữ. Cách phân tách này cho phép kiểm thử nghiệp vụ mà không phụ thuộc giao thức HTTP.

4.3.1 API quản lý

Bảng 4.2 trình bày các nhóm API chính. Đường dẫn thực tế cần được điều chỉnh theo mã nguồn nhưng phải duy trì tài nguyên và quyền tương ứng.

Bảng 4.2: Các nhóm API quản lý chính

Nhóm	Ví dụ tài nguyên	Chức năng
Devices	<code>/devices</code>	Đăng ký, lọc, lấy trạng thái và cập nhật nhãn thiết bị
Models	<code>/models</code>	Tạo phiên bản, tải manifest, kiểm tra và công bố gói mô hình
Deployments	<code>/deployments</code>	Lập kế hoạch, phê duyệt, theo dõi, dừng và rollback
Commands	<code>/commands</code>	Tạo lệnh đơn lẻ, xem vòng đời và kết quả
Telemetry	<code>/metrics</code>	Truy vấn chuỗi thời gian và tổng hợp theo nhóm thiết bị
Alerts	<code>/alerts</code>	Xem, xác nhận và đóng cảnh báo
Audit	<code>/audit-events</code>	Truy vết hành động theo actor, resource hoặc correlation ID
Agent tools	<code>/agent-tools</code>	Tập công cụ có schema dành cho AI Agent

Mọi yêu cầu ghi phải có correlation ID. API trả mã lỗi có cấu trúc gồm mã nghiệp vụ, thông điệp an toàn cho người dùng và chi tiết chẩn đoán dành cho log. Với thao tác tạo deployment, server tính lại phạm vi từ bộ lọc tại thời điểm phê duyệt, tránh sử dụng danh sách thiết bị đã cũ.

4.3.2 Xử lý heartbeat và reported state

Consumer nhận heartbeat từ broker, xác thực danh tính topic với device ID trong message và cập nhật thời điểm cuối. Một tác vụ nền đánh dấu thiết bị ngoại tuyến nếu không có heartbeat trong khoảng thời gian quy định. Reported state được cập nhật theo generation; bản tin cũ hơn generation đã biết bị bỏ qua. Khi reported state khác desired state quá thời hạn, hệ thống tạo cảnh báo hội tụ thất bại.

4.3.3 Bộ điều phối triển khai

Deployment Service chia phạm vi thành các batch. Mỗi batch có giới hạn song song và cửa sổ quan sát. Bộ điều phối chỉ chuyển sang batch kế tiếp khi tỷ lệ thành công, độ trễ và tỷ lệ lỗi suy luận vẫn trong ngưỡng. Nếu vi phạm,

deployment chuyển sang trạng thái tạm dừng. Người quản trị có thể tiếp tục, hủy hoặc tạo deployment rollback. Cơ chế này giúp hạn chế lỗi lan rộng từ mô hình hoặc cấu hình không phù hợp.

4.4 Hiện thực Dashboard

Dashboard truy cập dữ liệu qua API thay vì kết nối trực tiếp cơ sở dữ liệu. Dữ liệu gần thời gian thực được cập nhật qua kênh sự kiện; nếu kênh này mất, giao diện chuyển sang polling với tần suất thấp hơn. Bộ lọc trên URL giúp chia sẻ cùng một góc nhìn khi phối hợp xử lý sự cố.

Trang chi tiết thiết bị ưu tiên thông tin hành động được: trạng thái hiện tại, thời điểm liên lạc gần nhất, phiên bản mô hình, FPS, độ trễ, nhiệt độ và cảnh báo. Biểu đồ có đường đánh dấu thời điểm triển khai để liên hệ thay đổi phiên bản với biến động metric. Lịch sử lệnh trình bày người khởi tạo, nguồn khởi tạo là Dashboard hay AI Agent, tham số, kết quả và correlation ID.

Biểu mẫu triển khai thực hiện ba bước: chọn mô hình và phạm vi; xem kiểm tra tương thích và kế hoạch batch; xác nhận hoặc gửi phê duyệt. Với thiết bị không tương thích, giao diện hiển thị lý do cụ thể như runtime quá cũ, thiếu bộ nhớ hoặc không hỗ trợ precision yêu cầu.

4.5 Hiện thực AI Agent

AI Agent nhận danh tính và vai trò của người dùng từ lớp xác thực. Prompt hệ thống mô tả nhiệm vụ, giới hạn và yêu cầu trích dẫn dữ liệu vận hành trong câu trả lời. Danh sách công cụ được tạo theo quyền: người xem chỉ có công cụ đọc; người vận hành có thể tạo kế hoạch; công cụ phê duyệt chỉ xuất hiện với vai trò thích hợp.

Mỗi công cụ dùng schema để giới hạn trường và kiểu dữ liệu. Ví dụ, công cụ tạo kế hoạch triển khai chỉ nhận model version, bộ lọc thiết bị, batch size, ngưỡng dừng và lý do. Server tự xác định người dùng và không chấp nhận actor do mô hình ngôn ngữ cung cấp. Kết quả công cụ trả về mã tài nguyên và dữ liệu tối thiểu cần thiết, tránh đưa bí mật hoặc log thô quá dài vào ngữ cảnh.

Luồng thao tác được hiện thực theo hai pha. Pha lập kế hoạch chỉ đọc dữ liệu và tạo bản nháp có thời hạn. Pha thực thi yêu cầu người dùng xác nhận bản nháp; server kiểm tra lại quyền, trạng thái tài nguyên và chính sách rồi mới tạo deployment hoặc command. Khi công cụ lỗi, AI Agent phải phản ánh đúng trạng

thái, không được suy diễn rằng thao tác đã hoàn thành.

4.6 Đóng gói và triển khai

Control plane có thể được triển khai thành các dịch vụ container hoặc tiến trình độc lập tùy quy mô. Cơ sở dữ liệu và broker cần volume bền vững, sao lưu và health check. Edge Agent phù hợp với service manager của hệ điều hành để tự khởi động và giới hạn số lần khởi động lại. Phiên bản Agent và runtime được phát hành độc lập với phiên bản mô hình nhưng manifest phải khai báo tương thích.

Quy trình phát hành phần mềm gồm kiểm thử đơn vị, kiểm thử tích hợp, quét cấu hình, tạo bản dựng có phiên bản và triển khai môi trường thử trước khi lên môi trường chính. Migration cơ sở dữ liệu phải có thứ tự và phương án quay lui. Cấu hình quan sát như ngưỡng cảnh báo được quản lý bằng phiên bản để tránh khác biệt không truy vết được giữa các môi trường.

4.7 Kiểm thử cài đặt

Kiểm thử đơn vị bao phủ quy tắc chuyển trạng thái, kiểm tra manifest, RBAC và phân lô deployment. Kiểm thử tích hợp sử dụng Edge Agent mô phỏng để tạo heartbeat, nhận lệnh và báo cáo nhiều loại kết quả. Kiểm thử end-to-end bắt đầu từ Dashboard hoặc AI Agent, đi qua API và broker đến Agent, sau đó xác nhận audit trail đầy đủ.

Các ca lỗi quan trọng gồm tệp sai hàm băm, hết dung lượng, runtime không hỗ trợ phép toán, mất mạng trong khi tải, broker gửi trùng, Agent khởi động lại khi lệnh đang chạy, health check thất bại sau kích hoạt và người dùng phê duyệt kế hoạch đã hết hạn. Kết quả kiểm thử này được tổng hợp tại Chương V.

4.8 Kết chương

Chương IV đã mô tả cách hiện thực các mô-đun của kiến trúc đề xuất. Điểm trọng tâm là cập nhật mô hình nguyên tử, bộ đệm ngoại tuyến, điều phối triển khai theo batch, API có kiểm soát quyền và AI Agent theo hai pha lập kế hoạch–thực thi. Chương V tiếp tục xây dựng môi trường đo, kịch bản và bảng kết quả để đánh giá cả mô hình lẫn hệ thống vận hành.

CHƯƠNG V. THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày phương pháp thực nghiệm để trả lời ba câu hỏi: kỹ thuật tối ưu ảnh hưởng thế nào đến chất lượng và hiệu năng YOLO; hệ thống quản lý có vận hành tin cậy khi thiết bị và mạng gặp lỗi hay không; AI Agent có thực hiện đúng quyền, đúng phạm vi và đúng luồng phê duyệt hay không. Các ô kết quả được để trống có chủ đích nhằm tránh sử dụng số liệu chưa được đo. Trước khi nộp, sinh viên cần chạy thí nghiệm, điền số liệu, bổ sung biểu đồ và phân tích nguyên nhân.

5.1 Môi trường thực nghiệm

Toàn bộ biến thể mô hình phải được đánh giá trên cùng tập dữ liệu và cùng điều kiện thiết bị. Bảng 5.1 ghi cấu hình cần công bố để kết quả có thể tái lập.

Bảng 5.1: Cấu hình phần cứng và phần mềm thực nghiệm

Hạng mục	Giá trị
Thiết bị Edge AI	[CẦN BỔ SUNG: tên thiết bị, phiên bản phần cứng]
CPU/GPU/NPU	[CẦN BỔ SUNG: model, số lõi, bộ nhớ]
RAM và lưu trữ	[CẦN BỔ SUNG: dung lượng, loại lưu trữ]
Hệ điều hành	[CẦN BỔ SUNG: tên và phiên bản]
Runtime/SDK	[CẦN BỔ SUNG: tên và phiên bản]
Phiên bản Edge Agent	[CẦN BỔ SUNG: tag hoặc commit]
Máy chủ control plane	[CẦN BỔ SUNG: CPU, RAM, hệ điều hành]
Mạng thử nghiệm	[CẦN BỔ SUNG: loại mạng, băng thông, RTT]
Thiết bị đo công suất	[CẦN BỔ SUNG: tên thiết bị hoặc phương pháp]
Nhiệt độ môi trường	[CẦN BỔ SUNG: giá trị và cách đo]

5.2 Dữ liệu và mô hình cơ sở

Bộ dữ liệu bộ dữ liệu của đề tài phải được mô tả về nguồn gốc, giấy phép, số lớp, số ảnh và phân bố đối tượng. Việc chia tập được thực hiện trước khi tối ưu để tránh thay đổi tập kiểm tra giữa các thí nghiệm. Nếu dữ liệu là video, các khung hình gần nhau không nên bị chia ngẫu nhiên sang cả tập huấn luyện và kiểm tra vì

sẽ làm kết quả lạc quan giả tạo.

Bảng 5.2: Thống kê bộ dữ liệu

Tập dữ liệu	Số ảnh	Số đối tượng	Số lớp	Tỷ lệ
Huấn luyện	—	—	—	—
Xác thực	—	—	—	—
Kiểm tra	—	—	—	—

Mô hình cơ sở là mô hình YOLO được lựa chọn ở định dạng và precision ban đầu. Cần ghi rõ checkpoint, kích thước đầu vào, số epoch, optimizer, learning rate, augmentation, ngưỡng confidence và IoU. Nếu sử dụng checkpoint công khai, báo cáo phải nêu nguồn; nếu huấn luyện lại, cần lưu seed và cấu hình.

5.3 Chỉ số đánh giá

Nhóm chất lượng nhận diện gồm precision, recall, F1, mAP theo quy ước đã chọn và AP theo lớp. Nhóm hiệu năng gồm thời gian khởi động, độ trễ trung bình, P50, P95, P99, FPS bền vững, RAM đỉnh, mức sử dụng bộ tăng tốc, nhiệt độ và công suất. Kích thước tệp mô hình được báo cáo nhưng không thay thế các chỉ số chạy thực tế.

Nhóm vận hành gồm tỷ lệ thiết bị đăng ký thành công, độ trễ lệnh, tỷ lệ triển khai thành công, thời gian rollback, tỷ lệ message trùng được loại bỏ, thời gian phát hiện ngoại tuyến và mức mất telemetry. Nhóm AI Agent gồm tỷ lệ chọn đúng công cụ, tỷ lệ đúng tham số và phạm vi, tỷ lệ yêu cầu phê duyệt đúng, tỷ lệ từ chối thao tác vượt quyền và mức đầy đủ của audit trail.

5.4 Quy trình đo hiệu năng mô hình

Mỗi cấu hình được warm-up trước khi ghi số liệu. Sau đó chạy ít nhất **[CẦN BỔ SUNG: số vòng lặp]** vòng hoặc trong **[CẦN BỔ SUNG: thời gian]** phút. Cùng một luồng dữ liệu đầu vào được dùng cho các biến thể. Thiết bị được đưa về trạng thái nhiệt tương đương trước mỗi lượt, hoặc nhiệt độ được ghi để phân tích throttling. Mỗi thí nghiệm lặp lại ít nhất ba lần và báo cáo trung bình cùng độ lệch chuẩn.

Đo hai chế độ. Chế độ *model-only* chỉ đo runtime để xác định năng lực lõi.

Chế độ *end-to-end* bao gồm thu nhận, tiền xử lý, suy luận, hậu xử lý và công bố kết quả. Nếu hai giá trị chênh lệch lớn, cần tối ưu pipeline thay vì tiếp tục nén mạng.

5.5 Kết quả tối ưu mô hình

Bảng 5.3 và Bảng 5.4 là nơi ghi kết quả thực nghiệm. Các biến thể không được chạy phải xóa khỏi bảng; các biến thể khác có thể bổ sung nhưng phải mô tả đầy đủ tham số.

Bảng 5.3: Ảnh hưởng của tối ưu đến chất lượng nhận diện

Biến thể mô hình	Định dạng	P	R	F1	mAP 50	mAP 50–95	Δ mAP 50–95
Mô hình cơ sở	FP32	–	–	–	–	–	0,00
Cùng kiến trúc	FP16	–	–	–	–	–	–
Cùng kiến trúc	INT8–PTQ	–	–	–	–	–	–
Cùng kiến trúc	INT8–QAT	–	–	–	–	–	–
Biến thể rút gọn	–	–	–	–	–	–	–

Ghi chú: $\uparrow$ biểu thị giá trị càng lớn càng tốt. Cột Δ mAP 50–95 là mức thay đổi tuyệt đối so với mô hình cơ sở; giá trị âm thể hiện suy giảm chất lượng. Khi có số liệu, in đậm cấu hình tốt nhất trong nhóm đáp ứng yêu cầu triển khai.

Bảng 5.4: Hiệu năng mô hình trên thiết bị Edge AI

Biến thể mô hình	Định dạng	Kích thước (MB) $\downarrow$	P50 (ms) $\downarrow$	P95 (ms) $\downarrow$	FPS $\uparrow$	RAM (MB) $\downarrow$	Công suất (W) $\downarrow$	Tăng tốc ($\times$) $\uparrow$
Mô hình cơ sở	FP32	–	–	–	–	–	–	1,00
Cùng kiến trúc	FP16	–	–	–	–	–	–	–
Cùng kiến trúc	INT8–PTQ	–	–	–	–	–	–	–
Cùng kiến trúc	INT8–QAT	–	–	–	–	–	–	–
Biến thể rút gọn	–	–	–	–	–	–	–	–

Ghi chú: $\uparrow/\downarrow$ chỉ hướng cải thiện của từng chỉ số. Tăng tốc được tính theo P50 của mô hình cơ sở chia cho P50 của biến thể; mọi phép đo sử dụng cùng batch size và cùng điều kiện thiết bị.

Sau khi điền số liệu, phần phân tích cần trả lời ít nhất bốn vấn đề. Thứ nhất, cấu hình nào đạt ràng buộc chất lượng tối thiểu. Thứ hai, mức tăng tốc đến từ giảm phép tính hay từ backend hỗ trợ precision tốt hơn. Thứ ba, lớp hoặc kích thước vật thể nào bị ảnh hưởng nhiều nhất. Thứ tư, cấu hình có duy trì hiệu năng trong phép chạy dài hay bị giảm xung do nhiệt. Không nên kết luận “INT8 tốt nhất” chỉ vì có FPS cao nếu độ chính xác không đáp ứng yêu cầu.

5.6 Đánh giá chuỗi tiền xử lý và hậu xử lý

Để xác định nút thắt, thời gian end-to-end được phân rã theo Bảng 5.5. Kết quả này cho biết việc tối ưu runtime có còn ý nghĩa hay cần chuyển resize, chuyển màu, giải mã hoặc NMS sang bộ tăng tốc.

Bảng 5.5: Phân rã độ trễ chuỗi xử lý

Cấu hình	Thu nhận	Tiền xử lý	Suy luận	Hậu xử lý	Tổng (ms)
Cơ sở	—	—	—	—	—
Tối ưu được chọn	—	—	—	—	—

5.7 Đánh giá hệ thống quản lý Edge Agent

Control plane được kiểm thử với thiết bị thật kết hợp Agent mô phỏng. Agent mô phỏng cho phép tăng số lượng kết nối và tái tạo lỗi có kiểm soát, trong khi thiết bị thật xác nhận hành vi runtime. Bảng 5.6 liệt kê các kịch bản bắt buộc.

Bảng 5.6: Kết quả kiểm thử kịch bản vận hành

Mã	Kịch bản	Kết quả mong đợi	Kết quả
ST01	Đăng ký thiết bị mới	Cấp danh tính và xuất hiện trên Dashboard	—
ST02	Mất mạng rồi kết nối lại	Suy luận cục bộ tiếp tục; trạng thái và dữ liệu được đồng bộ lại	—
ST03	Broker gửi trùng lệnh	Lệnh chỉ được thực thi một lần	—
ST04	Tệp mô hình sai hàm băm	Từ chối kích hoạt; phiên bản hiện tại không đổi	—
ST05	Health check bản mới lỗi	Tự động rollback và tạo cảnh báo	—
ST06	Agent khởi động lại	Khôi phục cấu hình cuối và hội tụ desired state	—
ST07	Triển khai theo batch	Chỉ mở batch mới khi batch trước đạt ngưỡng	—

Bảng 5.6 (tiếp theo)

Mã	Kịch bản	Kết quả mong đợi	Kết quả
ST08	Tỷ lệ lỗi vượt ngưỡng	Deployment tự động tạm dừng	—
ST09	Thiết bị quá nhiệt	Chuyển degraded và sinh cảnh báo	—
ST10	Người dùng vượt quyền	API từ chối và ghi audit event	—

Khả năng chịu tải được đo bằng cách tăng số Agent mô phỏng theo bậc. Với mỗi mức, ghi tỷ lệ heartbeat xử lý thành công, độ trễ API, độ trễ phân phối lệnh, CPU, RAM của backend và broker. Ngưỡng công suất hệ thống là mức lớn nhất vẫn đáp ứng tiêu chí dịch vụ.

Bảng 5.7: Kết quả kiểm thử tải control plane

Số Agent	Msg/s	API P95 (ms)	Lệnh P95 (ms)	CPU (%)	RAM (MB)
10	–	–	–	–	–
50	–	–	–	–	–
100	–	–	–	–	–
200	–	–	–	–	–

5.8 Đánh giá AI Agent

Bộ đánh giá AI Agent gồm yêu cầu đọc, yêu cầu thao tác an toàn, yêu cầu mơ hồ, yêu cầu vượt quyền và nội dung chứa chỉ thị độc hại trong log. Mỗi trường hợp được chạy lặp với ngữ cảnh cố định. Kết quả được chấm theo rubric: chọn đúng công cụ, tham số đúng, phạm vi đúng, có giải thích dựa trên dữ liệu, có yêu cầu phê duyệt khi cần và không tuyên bố thành công khi công cụ lỗi.

Bảng 5.8: Kết quả đánh giá AI Agent

Nhóm yêu cầu	Số ca	Tỷ lệ đạt	Lỗi điển hình
Truy vấn trạng thái và metric	–	–	–
Chẩn đoán sau triển khai	–	–	–
Tạo kế hoạch cấu hình/triển khai	–	–	–
Yêu cầu mơ hồ cần hỏi lại	–	–	–
Thao tác vượt quyền	–	–	–
Prompt injection trong dữ liệu	–	–	–
Lỗi hoặc timeout công cụ	–	–	–

Đối với thao tác ghi, tiêu chí an toàn quan trọng hơn mức tự nhiên của câu trả lời. Một ca chỉ được xem là đạt khi Agent tạo đúng bản nháp, nêu rõ phạm vi, chờ phê duyệt và audit trail liên kết được người dùng với tài nguyên được tạo. Nếu Agent hỏi lại trong yêu cầu mơ hồ, đó là hành vi đúng chứ không phải thất bại.

5.9 Thảo luận kết quả

Sau khi có số liệu, phần này cần chọn cấu hình cuối dựa trên ràng buộc ứng dụng. Có thể sử dụng Bảng 5.9 để tránh lựa chọn chỉ theo một chỉ số. Trọng số phải được xác định trước và giải thích.

Bảng 5.9: Ma trận lựa chọn cấu hình triển khai

Cấu hình	Chất lượng	Độ trễ	Bộ nhớ	Công suất	Điểm tổng
Cơ sở	—	—	—	—	—
FP16	—	—	—	—	—
INT8	—	—	—	—	—
Biến thể rút gọn	—	—	—	—	—

Phần thảo luận cũng cần liên hệ kết quả mô hình với vận hành. Ví dụ, mô hình nhanh hơn có thể giảm nhiệt và cảnh báo, nhưng phiên bản runtime đặc thù có thể làm giảm tính tương thích giữa thiết bị. Tần suất telemetry quá cao giúp quan sát chi tiết nhưng tăng tải mạng và máy chủ. AI Agent rút ngắn thời gian truy vấn, song thao tác ghi phải chấp nhận thêm bước phê duyệt để bảo đảm an toàn.

5.10 Các nguy cơ ảnh hưởng tính hợp lệ

Tính hợp lệ nội tại có thể bị ảnh hưởng bởi nhiệt độ ban đầu, tiến trình nền, cache hoặc dữ liệu đầu vào khác nhau. Biện pháp giảm thiểu là lặp phép đo, cố định cấu hình và ghi telemetry. Tính hợp lệ ngoại tại bị giới hạn bởi số loại thiết bị và bộ dữ liệu; kết quả trên thiết bị Edge AI mục tiêu không tự động áp dụng cho mọi nền tảng. Tính hợp lệ của kết luận có thể bị ảnh hưởng nếu số lần chạy ít hoặc chỉ báo cáo trung bình; vì vậy cần báo cáo phân vị và độ biến thiên.

Đối với AI Agent, kết quả có thể thay đổi theo phiên bản mô hình, prompt và công cụ. Báo cáo phải lưu phiên bản, cấu hình suy luận và bộ ca đánh giá. Đánh giá an toàn không chứng minh hệ thống không còn rủi ro; nó chỉ cung cấp bằng chứng trong phạm vi kịch bản đã kiểm tra.

5.11 Kết chương

Chương V đã xây dựng phương pháp đánh giá đồng thời chất lượng nhận diện, hiệu năng Edge AI, độ tin cậy của control plane và mức an toàn của AI Agent.

Các bảng chưa có số liệu là danh sách công việc thực nghiệm bắt buộc, không phải kết quả. Khi hoàn thiện, sinh viên cần điền số đo, bổ sung biểu đồ và viết nhận xét dựa trên bằng chứng trước khi đưa ra kết luận cuối cùng.

KẾT LUẬN

Kết quả đạt được

Đồ án đã tiếp cận bài toán triển khai YOLO trên Edge AI theo góc nhìn toàn vòng đời thay vì chỉ tối ưu một tập mô hình. Báo cáo đã xác định các chỉ số chất lượng, hiệu năng và vận hành; xây dựng quy trình tạo đường chuẩn, tối ưu, kiểm chứng, đóng gói và phát hành mô hình. Quy trình này giúp mỗi phiên bản có manifest, hàm băm, thông tin tương thích và bằng chứng đánh giá trước khi triển khai.

Ở tầng hệ thống, đồ án đã thiết kế kiến trúc control plane–edge plane cho phép quản lý danh tính thiết bị, desired state, phiên bản mô hình, cấu hình, telemetry, cảnh báo và lệnh có vòng đời. Edge Agent hỗ trợ hoạt động khi mất kết nối, cập nhật nguyên tử, health check và rollback. Dashboard hợp nhất trạng thái thiết bị, chỉ số và lịch sử triển khai để người vận hành xác định nhanh vấn đề.

Đồ án cũng đã đưa AI Agent vào luồng điều khiển với các giới hạn rõ ràng. AI Agent chỉ được gọi công cụ có schema, nhận quyền theo người dùng và mặc định ở chế độ đọc. Các hành động ảnh hưởng hệ thống phải đi qua bước lập kế hoạch, kiểm tra chính sách, phê duyệt và audit. Thiết kế này khai thác lợi ích của giao tiếp ngôn ngữ tự nhiên nhưng vẫn duy trì cơ chế kiểm soát của hệ thống quản trị.

Các kết quả định lượng cuối cùng cần được tổng hợp từ Chương V sau khi hoàn tất đo trên thiết bị Edge AI mục tiêu. Báo cáo hoàn chỉnh phải nêu rõ cấu hình được lựa chọn, mức thay đổi mAP, độ trễ, FPS, bộ nhớ, công suất, tỷ lệ triển khai thành công và kết quả đánh giá AI Agent. **[CẦN BỔ SUNG: viết 1 đoạn 5–7 câu tóm tắt các con số thực nghiệm nổi bật và so sánh với mô hình cơ sở].**

Hạn chế

Phạm vi thiết bị và dữ liệu của đồ án còn giới hạn nên chưa đại diện cho mọi nền tảng Edge AI và môi trường hoạt động. Chất lượng mô hình có thể suy giảm khi phân bố dữ liệu thực tế khác tập đánh giá. Hệ thống quản lý mới tập trung vào các chức năng cốt lõi, chưa thay thế nền tảng MLOps quy mô lớn. AI Agent vẫn phụ thuộc vào chất lượng mô hình ngôn ngữ và bộ ca đánh giá; cơ chế phê duyệt làm giảm rủi ro nhưng không loại bỏ hoàn toàn khả năng diễn giải sai.

Hướng phát triển

Hướng phát triển thứ nhất là mở rộng kiểm thử trên nhiều kiến trúc phần cứng và tự động chọn gói mô hình phù hợp theo năng lực thiết bị. Hướng thứ hai là bổ sung học liên tục hoặc federated learning trong điều kiện dữ liệu được quản trị chặt chẽ. Hướng thứ ba là xây dựng phát hiện drift dựa trên thống kê đầu vào và phản hồi vận hành. Hướng thứ tư là tăng cường chuỗi cung ứng mô hình bằng ký số, chứng thực thiết bị và chính sách zero trust chi tiết hơn. Cuối cùng, AI Agent có thể được cải tiến bằng bộ đánh giá hồi quy tự động, mô phỏng sự cố và cơ chế đề xuất hành động dựa trên runbook đã được phê duyệt.

TÀI LIỆU THAM KHẢO

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.
- [2] J. Redmon and A. Farhadi, “YOLO9000: Better, Faster, Stronger,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 7263–7271.
- [3] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [4] B. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [5] S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *International Conference on Learning Representations*, 2016.
- [6] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” *arXiv preprint arXiv:1503.02531*, 2015.
- [7] ONNX Community. “Open Neural Network Exchange Documentation,” visited on Sept. 23, 2026. address: <https://onnx.ai/onnx/>
- [8] NVIDIA. “NVIDIA TensorRT Documentation,” visited on Sept. 23, 2026. address: <https://docs.nvidia.com/deeplearning/tensorrt/>
- [9] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016. DOI: 10.1109/JIOT.2016.2579198
- [10] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017. DOI: 10.1109/MC.2017.9
- [11] OASIS Open, “MQTT Version 5.0,” Organization for the Advancement of Structured Information Standards, tech. rep., 2019. visited on Sept. 23, 2026. address: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

- [12] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” National Institute of Standards and Technology, tech. rep. NIST Special Publication 800-207, 2020. DOI: 10.6028/NIST.SP.800-207

PHỤ LỤC A. GIAO DIỆN LẬP TRÌNH VÀ CẤU HÌNH THAM CHIẾU

A.1 Cấu trúc manifest của gói mô hình

Đoạn mã dưới đây minh họa manifest tối thiểu. Tên trường có thể thay đổi theo mã nguồn thực tế nhưng cần giữ đủ thông tin nhận dạng, kiểm tra toàn vẹn, tương thích và cấu hình suy luận.

Listing A.1: Ví dụ manifest của gói mô hình

```
{
  "schema_version": "1.0",
  "model_name": "yolo-edge-detector",
  "model_version": "1.2.0",
  "artifact": {
    "file": "model.engine",
    "sha256": "<sha256-value>",
    "size_bytes": 0
  },
  "runtime": {
    "backend": "<runtime-name>",
    "min_version": "<minimum-version>",
    "precision": "INT8"
  },
  "input": {
    "width": 640,
    "height": 640,
    "layout": "NCHW",
    "color": "RGB"
  },
  "postprocess": {
    "confidence_threshold": 0.25,
    "iou_threshold": 0.45
  },
  "labels_file": "labels.txt",
  "metrics": {
    "map_50": null,
    "map_50_95": null,
    "latency_p95_ms": null
  }
}
```

```
}
}
```

A.2 Ví dụ bản tin heartbeat

Listing A.2: Bản tin heartbeat của Edge Agent

```
{
  "schema_version": "1.0",
  "device_id": "edge-001",
  "sequence": 10241,
  "timestamp": "2026-09-23T13:18:14Z",
  "agent_version": "0.9.0",
  "health": "healthy",
  "reported_state": {
    "generation": 17,
    "model_version": "1.2.0",
    "config_version": "3.1.0"
  },
  "metrics": {
    "fps": 0.0,
    "latency_p95_ms": 0.0,
    "memory_used_mb": 0,
    "temperature_c": 0.0
  }
}
```

A.3 Ví dụ tạo kế hoạch triển khai

Yêu cầu tạo kế hoạch chỉ tạo bản nháp, chưa thay đổi desired state. Máy chủ tự lấy danh tính người dùng từ phiên đăng nhập.

Listing A.3: Yêu cầu tạo kế hoạch triển khai

```
POST /agent-tools/create-deployment-plan
{
  "model_version": "1.2.0",
  "device_filter": {
    "site": "site-a",
    "hardware": "<edge-hardware>"
  }
}
```

```
} ,
"strategy": {
  "batch_size": 5,
  "max_failure_rate": 0.1,
  "observation_seconds": 300
},
"reason": "Reduce inference latency"
}
```

Phản hồi gồm mã kế hoạch, danh sách hoặc số lượng thiết bị, các thiết bị không tương thích, rủi ro và thời hạn. Công cụ phê duyệt nhận mã kế hoạch thay vì nhận lại toàn bộ tham số, nhờ đó hạn chế việc AI Agent thay đổi phạm vi giữa hai bước.

A.4 Ma trận quyền tham khảo

Bảng A.1: Ma trận quyền theo vai trò

Thao tác	Người xem	Vận hành	Kỹ sư mô hình	Quản trị
Xem thiết bị và metric	Có	Có	Có	Có
Xác nhận cảnh báo	Không	Có	Có	Có
Tạo lệnh cục bộ	Không	Có	Có	Có
Tải gói mô hình	Không	Không	Có	Có
Tạo kế hoạch triển khai	Không	Có	Có	Có
Phê duyệt triển khai diện rộng	Không	Không	Không	Có
Quản lý người dùng và vai trò	Không	Không	Không	Có
Xem audit log	Giới hạn	Giới hạn	Giới hạn	Có

PHỤ LỤC B. NHẬT KÝ THỰC HIỆN ĐATN

Mẫu ĐATN-02

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ
NỘI
KHOA CÔNG NGHỆ THÔNG TIN

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA
VIỆT NAM
Độc lập – Tự do – Hạnh phúc

NHẬT KÝ THỰC HIỆN ĐATN

*(Ban hành kèm theo Quyết định số 483/QĐ-ĐHXDHN ngày 20 tháng 3 năm 2026 của Hiệu trưởng
Trường Đại học Xây dựng Hà Nội)*

Họ và tên SV:	Phạm Công Quân	Mã SV:	0201366
Tên đề tài:	Xây dựng và phát triển hệ thống Game Engine	Đợt:	D2
GVHD:	Trần Khánh Dung	Đơn vị:	Công nghệ phần mềm

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/ kỹ năng áp dụng	Các vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Tuần 25: 19/01/2026 – 25/01/2026							
Làm việc độc lập	10 giờ	- Nhận đề tài ĐATN: xây dựng Game Engine 3D bằng C++ - Nghiên cứu kiến trúc game engine (Core, Rendering, ECS) - Thiết lập môi trường: Visual Studio, CMake, C++20 - Tạo cấu trúc dự án CMake đa module - Thiết lập môi trường phát triển: Visual Studio, CMake, C++20	- Cấu trúc thư mục: engine/code/, engine/libs/, game/ - CMakeLists.txt cho 3 module: libs, engine (static lib), game (executable) - Tích hợp thư viện: GLFW, GLAD, GLM, ImGui, stb, tinyglTF, assimp	- Kiến trúc game engine: vòng lặp game, rendering, quản lý tài nguyên - CMake cho dự án C++ đa module	- Tìm hiểu thêm sự khác biệt OpenGL và Vulkan - Xác định rõ phạm vi đề tài	CMake đa module giúp quản lý dự án lớn hiệu quả	Sinh viên GVHD/ đồng HD
Tuần 26: 26/01/2026 – 01/02/2026							
Làm việc độc lập	15 giờ	- Xây dựng Core Engine: Engine class (Init, Run, Shutdown) - Triển khai Backend: cửa sổ GLFW, khởi tạo OpenGL qua GLAD - Thiết kế GameLayer interface - Xây dựng Logger, Common utilities, smart pointer types	- Core module: vòng lặp game (BeginFrame, Update, EndFrame) - Backend: tạo cửa sổ, khởi tạo OpenGL context - GameLayer interface cho game code plug vào engine	- Design pattern: Game Loop, Layer Architecture - C++20: smart pointers, RAII, template - Quản lý OpenGL context qua GLFW/GLAD	- Thêm xử lý lỗi cho quá trình khởi tạo - Tối ưu delta time	Layer Architecture tách biệt engine/game code hiệu quả	Sinh viên GVHD/ đồng HD

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/ kỹ năng áp dụng	Các vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Tuần 27: 02/02/2026 – 08/02/2026							
Làm việc độc lập	18 giờ	- Xây dựng API Layer: OpenGL Shader, Mesh, Texture, Renderer - Triển khai Material system PBR (diffuse, specular, normal, roughness, AO, emissive) - Viết GLSL shaders: standard.glsl, simple.glsl, shadow_depth.glsl	- OpenGL API: biên dịch shader, VAO/VBO/EBO, texture sRGB - Material PBR hoạt động (albedo, metallic, roughness...) - 3 shader programs hoàn chỉnh - Renderer với BeginFrame/EndFrame	- OpenGL pipeline: Vertex/Fragment shader, uniform, texture binding - PBR và các thành phần vật liệu - GLSL shader language	- Hỗ trợ thêm texture format (HDR, compressed) - Tối ưu batch rendering	- PBR phức tạp nhưng cho kết quả trực quan tốt - Lỗi shader khó phát hiện, cần debug cẩn thận	Sinh viên GVHD/đồng HD
Tuần 28: 09/02/2026 – 15/02/2026							
Làm việc độc lập	16 giờ	- Xây dựng Camera system: FPS camera, free-fly, Euler angles, Frustum Culling - Triển khai Input module: bàn phím/chuột qua GLFW - Xây dựng Light system: Directional, Point, Spot + ShadowMap	- Camera: tốc độ 7.0 units/sec, sprint 2.5x, sensitivity 0.0018 rad/pixel - Input: bắt sự kiện phím/chuột, cursor lock/unlock - 3 loại đèn + ShadowMap đổ bóng thời gian thực	- Hệ tọa độ 3D, Euler angles, ma trận view/projection - Xử lý input real-time - Mô hình chiếu sáng Blinn-Phong, Shadow Mapping	- Tối ưu shadow map resolution - Xử lý shadow acne và peter panning	- Frustum culling tăng hiệu suất đáng kể - Shadow mapping cần nhiều tinh chỉnh	Sinh viên GVHD/đồng HD
Tuần 29: 16/02/2026 – 22/02/2026							
NGHỈ TẾT NGUYÊN ĐÁN							
Tuần 30: 23/02/2026 – 01/03/2026							
NGHỈ TẾT NGUYÊN ĐÁN							

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/ kỹ năng áp dụng	Các vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Tuần 31: 02/03/2026 – 08/03/2026							
Làm việc độc lập	18 giờ	- Xây dựng Model loading: OBJ, glTF/GLB (tinyglTF), FBX (assimp) - Triển khai Asset Manager với caching - Xây dựng Scene Manager + SceneSerializer	- Tải thành công: Cubes.obj, Barbarian.glb, UAL1_Standard.fbx, scene.glTF - Asset Manager: cache tài nguyên, tránh tải trùng - Scene Manager: quản lý objects/lights, lưu/tải scene	- Các định dạng 3D (OBJ, glTF, FBX) - Design pattern: Flyweight (caching), Factory (loading) - Serialization: lưu/phục hồi trạng thái scene	- Xử lý edge case khi tải model (thiếu texture, material lỗi) - Thêm hỗ trợ animation data	- Caching tăng hiệu suất khi nhiều object dùng chung model - glTF hiệu quả hơn OBJ	Sinh viên GVHD/ đồng HD
Tuần 32: 09/03/2026 – 15/03/2026							
Làm việc độc lập	18 giờ	- Xây dựng ECS: Tag, Transform, Mesh, Material, Light, Camera Component - Triển khai PostProcess (Bloom, Tone mapping), Skybox, Terrain - Xây dựng Particle System và WorldGrid LOD	- ECS: 6 component types, entity management - PostProcessing: bloom, HDR tone mapping - Skybox, Terrain, Particle system với presets - WorldGrid với Level-of-Detail tự động	- ECS pattern: tách biệt data và logic - Post-Processing: framebuffer, multi-pass rendering - LOD cho tối ưu hiệu suất	- ECS cần thêm System layer hoàn chỉnh - Tối ưu particle system cho số lượng lớn	- ECS giúp code dễ mở rộng và bảo trì - Post-processing nâng cao chất lượng hình ảnh	Sinh viên GVHD/ đồng HD

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/ kỹ năng áp dụng	Các vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Tuần 33: 16/03/2026 – 22/03/2026							
Làm việc độc lập	18 giờ	- Xây dựng Editor UI (ImGui): scene panel, properties panel - Triển khai TransformGizmo (move/rotate/scale) và RaycasterPicker - Xây dựng DemoGame: FPS camera, cube, ground plane, lighting	- EditorUI ImGui hoàn chỉnh, Transform Gizmo 3 trục, RaycasterPicker - DemoGame: camera FPS, điều khiển cube, ground 50x50, directional + ambient light	- ImGui cho editor tools - Ray casting và intersection testing - Tích hợp module thành ứng dụng hoàn chỉnh - Vẽ sơ đồ UML và kiến trúc phần mềm	- Hoàn thiện editor (undo/redo, multi-select) - Viết thuyết minh ĐATN	- Editor tools tăng tốc phát triển game - Tài liệu hóa kiến trúc sớm giúp review dễ hơn	Sinh viên GVHD/ đồng HD
Tuần 34: 23/03/2026 – 29/03/2026							
Làm việc độc lập	15 giờ	- Bug fixing và tối ưu hiệu suất rendering - Sửa lỗi các module ECS, PostProcessing, Particle System sau quá trình test - Tiếp tục phát triển module mới: bổ sung Advanced ECS System layer	- Hệ thống ổn định, sẵn sàng cho buổi kiểm tra tiến độ - Cải thiện hiệu suất particle system và LOD sau tối ưu	- Testing, debugging và tối ưu hệ thống lớn	- Vulkan API, Animation System, Physics Engine - Hoàn thiện Advanced ECS features (System layer, Event bus)	- Animation và Physics là hai module trọng tâm cần ưu tiên trong M2 - Kết quả kiểm tra tiến độ là cơ sở để điều chỉnh kế hoạch phát triển phù hợp hơn	Sinh viên GVHD/ đồng HD
Tuần 35: 30/03/2026 – 05/04/2026							
Làm việc độc lập	18 giờ	- Tái cấu trúc engine sang lớp trừu tượng đồ họa RHI (đặt tên Vortex) - Thiết kế giao diện Vulkan khởi tạo IGraphicsDevice/ ICommandList độc lập API - Khởi tạo backend Vulkan: instance, physical/logical device, queue	- Bộ giao diện RHI: buffer, texture, sampler, bind group, pipeline - Backend Vulkan khởi tạo được device và swapchain cơ bản - Mô hình tài nguyên định danh bằng handle	- Vulkan API: device, queue, swapchain - Thiết kế API trung lập backend - Mô hình tài nguyên handle thay cho con trỏ thô	- Chuyển toàn bộ mã OpenGL cũ sang RHI - Chuẩn hóa vòng đời tài nguyên GPU	- Trừu tượng hóa RHI giúp thay backend mà không sửa mã cấp cao - Vulkan tường minh hơn OpenGL nên cần thiết kế cẩn thận	Sinh viên GVHD/ đồng HD

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/ kỹ năng áp dụng	Các vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Tuần 36: 06/04/2026 – 12/04/2026							
Làm việc độc lập	18 giờ	- Hoàn thiện backend Vulkan: command buffer, render pass, đồng bộ (fence, semaphore) - Quản lý pipeline barrier và chuyển layout ảnh theo trạng thái tài nguyên - Thiết kế ngân sách bind group 4 tập	- Vòng đời một khung hình ở tầng RHI chạy ổn định - Backend tự chèn barrier, mã cấp cao chỉ khai báo đọc/ghi - Vẽ được tam giác và mesh cơ bản qua RHI	- Đồng bộ GPU/CPU trong Vulkan - Descriptor set và bind group - Quản lý trạng thái tài nguyên	- Tối ưu số lần chuyển layout - Bổ sung kiểm tra validation layer	- Đưa việc đồng bộ vào backend giúp mã gọi đơn giản và ít lỗi hơn - Ngân sách 4 bind group cần cân nhắc kỹ từ đầu	Sinh viên
							GVHD/ đồng HD
Tuần 37: 13/04/2026 – 19/04/2026							
Làm việc độc lập	16 giờ	- Xây dựng phân hệ Core: math (Vec, Mat, Quat), Log, Settings, Diagnostics, bộ nhớ - Xây dựng phân hệ Jobs: thread pool, parallelFor, Channel - Chuẩn hóa quy ước tọa độ và ma trận chiếu	- Core cung cấp toán học, đo đạc và cấu hình cho toàn engine - Jobs chạy song song tác vụ nền, Channel bàn giao kết quả an toàn - Ví dụ math_primitives, diagnostics, async_tasks tự kiểm tra	- Thư viện toán học cho đồ họa - Lập trình đa luồng, hàng đợi MPMC - Chuỗi đo hiệu năng (profiler)	- Bổ sung thêm kiểm thử biên cho toán học - Tối ưu tranh chấp khóa trong Channel	- Tách Core và Jobs làm nền giúp các phân hệ trên kế thừa dễ dàng - Channel xác định rõ quyền sở hữu dữ liệu giữa các luồng	Sinh viên
							GVHD/ đồng HD

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/ kỹ năng áp dụng	Các vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Tuần 38: 20/04/2026 – 26/04/2026							
Làm việc độc lập	18 giờ	- Xây dựng ECS hướng dữ liệu theo cấu trúc sparse-set - Triển khai Registry, View, Event/Observer, CommandBuffer - Hệ thống biến đổi phân cấp và tuần tự hóa scene ra JSON	- Registry lưu component O(1), duyệt liên mạch thân thiện cache - View duyệt entity theo tập component; Command Buffer trì hoãn thay đổi cấu trúc - Lưu/nạp scene JSON, picking 2D/3D	- Kiến trúc ECS data-oriented - Sparse-set và swap-and-pop - Serialization tham chiếu tài nguyên qua đường dẫn	- Bổ sung hook vòng đời component - Kiểm thử duyệt an toàn khi cấu trúc thay đổi	- Sparse-set đạt được cả tra cứu O(1) lẫn duyệt tuần tự - Ghi texture theo đường dẫn giúp scene độc lập trạng thái GPU	Sinh viên
							GVHD/ đồng HD
Tuần 39: 27/04/2026 – 03/05/2026							
Làm việc độc lập	18 giờ	- Xây dựng đường kết xuất 2D: SpriteBatch gom draw call theo texture/sampler/blend - Triển khai Atlas đóng gói texture và 9-slice - Loại bỏ sprite ngoài màn hình ở bước trích xuất	- SpriteBatch vẽ hàng nghìn sprite trong ít lệnh vẽ - Atlas gộp nhiều PNG thành một texture; 9-slice co giãn không méo góc - Ví dụ sprites, sprite2d, atlas, ui	- Instancing và giảm số draw call - Đóng gói atlas, padding chống rỉ màu - Culling bằng hộp bao affine	- Tối ưu sắp xếp theo lớp - Bổ sung nhiều chế độ trộn	- Điều kiện gộp (texture, sampler, blend) quyết định hiệu năng 2D - Atlas tác động trực tiếp tới số lệnh vẽ	Sinh viên
							GVHD/ đồng HD
Tuần 40: 04/05/2026 – 10/05/2026							
Làm việc độc lập	18 giờ	- Xây dựng Tilemap nhiều lớp + autotile blob 47 hình dạng - Nhập bản đồ từ Tiled theo quy ước GID - Ánh sáng 2D, hệ hạt, bloom 2D và camera controller	- Autotile 47-blob suy ra tự động từ tileset, có bước tự kiểm tra - Bộ đệm ánh sáng 2D (cộng đèn, nhân bóng); hệ hạt pool cố định; bloom - Ví dụ tilemap, tiled, bloom2d, particles	- Autotile và mặt nạ viền - Quy ước GID của Tiled - Ánh sáng 2D theo bộ đệm riêng	- Bổ sung parallax cho lớp tilemap - Tối ưu độ phân giải bộ đệm ánh sáng	- Biến quy ước ngầm (47 mặt nạ) thành điều kiện kiểm chứng được - Tách bit gameplay khỏi đường vẽ giúp mã rõ ràng	Sinh viên
							GVHD/ đồng HD

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/ kỹ năng áp dụng	Các vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Tuần 41: 11/05/2026 – 17/05/2026							
Làm việc độc lập	18 giờ	- Xây dựng đường kết xuất 3D forward và MeshRenderer - Triển khai vật liệu PBR metallic–roughness (GGX) - Hình học nguyên thủy dựng sẵn và nạp mô hình glTF	- MeshRenderer vẽ mesh có ánh sáng theo PBR - Năm bản đồ (albedo, normal, metallic–roughness, emissive, occlusion) đều tùy chọn - Ví dụ mesh3d, gizmos3d, scene_viewer	- Mô hình PBR metallic–roughness, phản xạ GGX - Nạp glTF, primitive với pháp tuyến/tiếp tuyến - Ba loại đèn (directional, point, spot)	- Bổ sung chế độ deferred - Tối ưu số biến thể pipeline	- Lấp slot texture trống bằng texture trung tính giúp tránh nhiều biến thể shader - Winding CCW quyết định hướng pháp tuyến	Sinh viên GVHD/ đồng HD
Tuần 42: 18/05/2026 – 24/05/2026							
Làm việc độc lập	18 giờ	- Xây dựng chế độ deferred và render graph điều phối pass - Triển khai bản đồ bóng phân tầng (CSM) và SSAO - Tự động chèn barrier, sắp thứ tự pass theo phụ thuộc	- Render graph gom G-buffer, chèn barrier, sắp pass tự động - CSM 4 cascade trên một texture 2×2; SSAO có pass làm mờ riêng - Bóng mềm PCF bán kính cầu hình được	- Deferred shading và G-buffer - Cascaded shadow map, splitLambda - SSAO lấy mẫu độ sâu/pháp tuyến	- Tối ưu độ phân giải shadow map - Bổ sung contact shadow	- Render graph giúp thêm/bớt pass mà không sửa mã đồng bộ - Chia cascade cân bằng giữa phủ xa và chi tiết gần	Sinh viên GVHD/ đồng HD

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/ kỹ năng áp dụng	Các vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Tuần 43: 25/05/2026 – 31/05/2026							
Làm việc độc lập	16 giờ	- Bổ sung hiệu ứng dựa trên G-buffer: sương mù thể tích, decal, contact shadow - Ánh sáng dựa trên ảnh (IBL), khúc xạ (transmission), nướng lightmap, morph target - Xây dựng bộ nhớ đệm pipeline	- IBL từ cubemap irradiance/ environment; transmission khúc xạ cảnh phía sau - Lightmap baking, morph target, decal theo khối hộp - Pipeline cache giảm khựng khi biên dịch shader	- IBL, khúc xạ và thứ tự pass - Lightmap baking và tập UV thứ hai - Quản lý tổ hợp biến thể pipeline	- Tối ưu chi phí sương mù thể tích - Bổ sung kiểm chứng hình ảnh cho từng hiệu ứng	- Các hiệu ứng deferred làm giàu chất lượng nhưng cần cân đối chi phí - Cache pipeline cải thiện trải nghiệm rõ rệt	Sinh viên GVHD/ đồng HD
Tuần 44: 01/06/2026 – 07/06/2026							
Làm việc độc lập	18 giờ	- Xây dựng phân hệ anim: skeleton, clip, sampler tạo pose - Triển khai BlendTree (Clip/ Blend/Mask), máy trạng thái, root motion, sự kiện animation - Skinning hai backend CPU/GPU	- Luồng animation: state machine → clip → sampler → skeleton → skinning - BlendTree hòa trộn nhiều clip, Mask ghi đè theo khớp - Ví dụ anim_state, skinned tự kiểm tra	- Hệ animation, hòa trộn pose - Root motion tách chuyển động gốc - Skinning và ma trận da đẩy lên GPU	- Hoàn thiện phát animation mesh có xương qua vòng lặp App - Bổ sung crossfade và exit time	- Ba loại nút Clip/Blend/ Mask đủ mô tả mọi tình huống hòa trộn - Tách backend skinning theo môi trường tăng tính linh hoạt	Sinh viên GVHD/ đồng HD

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/ kỹ năng áp dụng	Các vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Tuần 45: 08/06/2026 – 14/06/2026							
Làm việc độc lập	18 giờ	- Xây dựng Physics bọc Box2D và tích hợp ECS - Xây dựng Text UTF-8 (stb_truetype) hỗ trợ tiếng Việt - Xây dựng UI immediate-mode 9-slice và Audio an toàn khi không có thiết bị	- Physics: RigidBody2D, collider, sensor, raycast, sự kiện va chạm - Text duyệt theo codepoint, từ chối overlong/surrogate, thay bằng U+FFFD - UI panel/nút/nhãn; Audio chạy được cả khi thiếu thiết bị	- Tích hợp Box2D (pixelsPer Meter, fixed step) - Giải mã UTF-8 phòng vệ - Immediate-mode GUI	- Bổ sung widget UI nâng cao (nhập liệu, cuộn) - Tích hợp âm thanh không gian	- Ghi đè bước cố định của Box2D bằng App tránh trôi lệch - Bộ giải mã UTF-8 phòng vệ giúp phát hiện lỗi sớm	Sinh viên
							GVHD/ đồng HD
Tuần 46: 15/06/2026 – 21/06/2026							
Làm việc độc lập	18 giờ	- Xây dựng AssetManager nạp bắt đồng bộ, glTF và hot-reload - Xây dựng vòng lặp App: hook, plugin, OverlayPlugin, chụp khung hình - Triển khai threaded Simulation (mô phỏng trước renderer một khung)	- load() trả handle tức thì; đọc/giải mã ở luồng IO, tải GPU ở luồng chính - App điều phối khung hình, game đăng ký callback qua hook - OverlayPlugin (F1), screenshot ra PNG, hot-reload	- Nạp tài nguyên bắt đồng bộ - Kiến trúc hook/plugin - Mô hình hai luồng game/main và hợp đồng sở hữu	- Tối ưu bàn giao dữ liệu giữa hai luồng - Bổ sung inspector runtime	- Nạp bắt đồng bộ tránh khựng khi vào khu vực mới - App che giấu phần lớn độ phức tạp khỏi mã game	Sinh viên
							GVHD/ đồng HD

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/ kỹ năng áp dụng	Các vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Tuần 47: 22/06/2026 – 28/06/2026							
Làm việc độc lập	16 giờ	- Xây dựng backend đồ họa thứ hai WebGPU (wgpu-native) - Cho phép chọn backend lúc chạy qua VORTEX_RHI_API - Kiểm chứng cùng mã renderer chạy trên Vulkan và WebGPU	- Backend WebGPU chạy được đường 2D và 3D - Cùng mã cấp cao hoạt động trên cả hai backend - Bộ điều khiển camera Orbit/Fly/Pan và ScreenShake	- WebGPU và sự khác biệt với Vulkan - Thiết kế API trung lập backend - Xử lý khác biệt sampler/binding giữa hai backend	- Hoàn thiện các tính năng còn khác biệt giữa hai backend - Bổ sung kiểm thử chéo backend	- Trừu tượng RHI chứng minh giá trị khi thêm backend thứ hai - Khác biệt sampler/descriptor cần xử lý riêng từng backend	Sinh viên
							GVHD/ đồng HD
Tuần 48: 29/06/2026 – 05/07/2026							
Làm việc độc lập	18 giờ	- Xây dựng trò chơi minh họa Farm RPG (2D) và Roller (3D) - Bổ sung game hoàn chỉnh breakout và alien_cake_addict - Viết khoảng 55 ví dụ theo quy ước self-check	- Farm RPG (khoảng 2.700 dòng): tilemap, autotile, ngày–đêm, lưu/ nạp, vật lý - Roller (khoảng 500 dòng): PBR, bóng thời gian thực, HUD, không thao tác RHI trực tiếp - Khoảng 55 ví dụ chạy headless qua VORTEX_MAX_FRAMES	- Tổng hợp các phân hệ thành trò chơi hoàn chỉnh - Thiết kế API tăng ứng dụng gọn - Quy ước ví dụ vừa là tài liệu vừa là kiểm thử	- Bổ sung nội dung gameplay - Chuẩn hóa log cho kiểm thử khởi động	- Roller khoảng 500 dòng cho thấy App che giấu tốt độ phức tạp kết xuất 3D - Ví dụ theo mã thoát tích hợp thẳng vào CI	Sinh viên
							GVHD/ đồng HD

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/ kỹ năng áp dụng	Các vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Tuần 49: 06/07/2026 – 12/07/2026							
Làm việc độc lập	18 giờ	- Hiện thực quy ước kiểm thử tự động và đo hiệu năng bằng benchmark - Xây dựng trang web giới thiệu và tài liệu (Next.js, Fumadocs, MDX, i18n) - Bug fixing toàn hệ thống theo kết quả kiểm thử	- Bộ self-check đều trả mã thoát 0; benchmark đo tới 200.000 sprite - Trang web sinh tĩnh: giới thiệu, tài liệu, blog, hỗ trợ Anh-Việt - Phát hiện và sửa các lỗi hồi quy (ánh sáng 2D, sự kiện animation, UTF-8)	- Đo và đánh giá hiệu năng - Next.js, Fumadocs, monorepo - pnpm - Testing hồi quy và ổn định hệ thống	- Bổ sung số liệu benchmark trên nhiều cấu hình - Hoàn thiện nội dung tài liệu	- Đạt 938 FPS ở 10.000 sprite, vượt mục tiêu 60 FPS - Quy ước kiểm thử phát hiện lỗi khó thấy bằng quan sát thủ công	Sinh viên GVHD/ đồng HD
Tuần 50: 13/07/2026 – 19/07/2026							
Làm việc độc lập	16 giờ	- Hoàn thiện thuyết minh đồ án tốt nghiệp và bộ hình minh họa - Đóng gói mã nguồn, viết README và tài liệu sử dụng - Rà soát tổng thể, chuẩn bị bảo vệ	- Thuyết minh hoàn chỉnh: phân tích, thiết kế, xây dựng, kết quả - Mã nguồn khoảng 28.000 dòng lõi, 14 phân hệ, hai backend, kèm README - Slide và demo chuẩn bị cho buổi bảo vệ	- Viết báo cáo khoa học - Tổng hợp và trình bày kết quả - Đóng gói và tài liệu hóa dự án	- Chỉnh sửa theo góp ý của GVHD - Rà soát lần cuối hình, bảng, tài liệu tham khảo	- Tài liệu hóa xuyên suốt giúp tổng hợp báo cáo thuận lợi - Đối chiếu kết quả với mục tiêu cho thấy đề tài đạt và vượt yêu cầu	Sinh viên GVHD/ đồng HD